

铁路知识检索增强智能问答软件源代码

软件名称：铁路知识检索增强智能问答软件 V1.0

版本号：V1.0

文档类型：源代码

目录

1. backend/alembic/env.py
2. backend/app/__init__.py
3. backend/app/api/__init__.py
4. backend/app/api/router.py
5. backend/app/api/routes/__init__.py
6. backend/app/api/routes/chat.py
7. backend/app/api/routes/health.py
8. backend/app/api/routes/terminology.py
9. backend/app/core/__init__.py
10. backend/app/core/config.py
11. backend/app/db/__init__.py
12. backend/app/db/base.py
13. backend/app/db/session.py
14. backend/app/main.py
15. backend/app/models/__init__.py
16. backend/app/models/entities.py
17. backend/app/rag/__init__.py
18. backend/app/rag/docx_loader.py
19. backend/app/rag/embedding.py
20. backend/app/rag/retriever.py
21. backend/app/rag/terminology_extractor.py
22. backend/app/rag/types.py
23. backend/app/rag/vector_store.py
24. backend/app/schemas/__init__.py
25. backend/app/schemas/chat.py
26. backend/app/schemas/health.py
27. backend/app/schemas/terminology.py
28. backend/app/services/__init__.py
29. backend/app/services/chat_service.py
30. backend/app/services/language.py
31. backend/app/services/llm_client.py
32. backend/app/services/terminology_service.py
33. backend/Dockerfile
34. backend/pyproject.toml
35. backend/railway_qa_agent_backend.egg-info/dependency_links.txt
36. backend/railway_qa_agent_backend.egg-info/requires.txt

backend/alembic/env.py

```
import asyncio
from logging.config import fileConfig

from sqlalchemy import pool
from sqlalchemy.engine import Connection
from sqlalchemy.ext.asyncio import async_engine_from_config

from alembic import context
from app import models # noqa: F401
from app.core.config import settings
from app.db.base import Base

config = context.config
config.set_main_option("sqlalchemy.url", settings.database_url)

if config.config_file_name is not None:
    fileConfig(config.config_file_name)

target_metadata = Base.metadata


def run_migrations_offline() -> None:
    context.configure(
        url=settings.database_url,
        target_metadata=target_metadata,
        literal_binds=True,
        dialect_opts={"paramstyle": "named"},
    )

    with context.begin_transaction():
        context.run_migrations()


def do_run_migrations(connection: Connection) -> None:
    context.configure(connection=connection, target_metadata=target_metadata)

    with context.begin_transaction():
        context.run_migrations()


async def run_async_migrations() -> None:
    connectable = async_engine_from_config(
        config.get_section(config.config_ini_section, {}),
        prefix="sqlalchemy.",
        poolclass=pool.NullPool,
    )

    async with connectable.connect() as connection:
        await connection.run_sync(do_run_migrations)

    await connectable.dispose()


def run_migrations_online() -> None:
    asyncio.run(run_async_migrations())


if context.is_offline_mode():
    run_migrations_offline()
else:
    run_migrations_online()
```

backend/app/__init__.py

```
"""Railway QA Agent backend package."""
```

backend/app/api/__init__.py

```
"""API routers."""
```

backend/app/api/router.py

```
from fastapi import APIRouter

from app.api.routes import chat, health, terminology

api_router = APIRouter()
api_router.include_router(health.router, tags=["health"])
api_router.include_router(chat.router, prefix="/chat", tags=["chat"])
api_router.include_router(terminology.router, prefix="/terminology", tags=["terminology"])
```

backend/app/api/routes/__init__.py

```
"""Route modules."""
```

backend/app/api/routes/chat.py

```
from typing import Annotated
from uuid import UUID

from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy.orm import selectinload

from app.db.session import get_db
from app.models import ChatMessage, ChatSession
from app.schemas import ChatMessageRead, ChatRequest, ChatResponse, ChatSessionRead
from app.services.chat_service import ChatService

router = APIRouter()
DbSession = Annotated[AsyncSession, Depends(get_db)]

@router.post("", response_model=ChatResponse)
async def chat(request: ChatRequest, db: DbSession) -> ChatResponse:
    return await ChatService(db).handle(request)

@router.get("/sessions", response_model=list[ChatSessionRead])
async def list_sessions(db: DbSession) -> list[ChatSession]:
    result = await db.execute(select(ChatSession).order_by(ChatSession.updated_at.desc()).limit(50))
    return list(result.scalars().all())

@router.get("/sessions/{session_id}", response_model=list[ChatMessageRead])
async def get_session_messages(
    session_id: UUID,
    db: DbSession,
) -> list[ChatMessage]:
    result = await db.execute(
        select(ChatSession)
    )
```

```

        .where(ChatSession.id == session_id)
        .options(selectinload(ChatSession.messages))
    )
    session = result.scalar_one_or_none()
    if session is None:
        raise HTTPException(status_code=404, detail="Chat session not found")
    return session.messages

```

backend/app/api/routes/health.py

```

from fastapi import APIRouter

from app.core.config import settings
from app.schemas import HealthResponse

router = APIRouter()

@router.get("/health", response_model=HealthResponse)
async def health() -> HealthResponse:
    return HealthResponse(status="ok", app=settings.app_name, environment=settings.app_env)

```

backend/app/api/routes/terminology.py

```

from typing import Annotated

from fastapi import APIRouter, Depends, Query
from sqlalchemy.ext.asyncio import AsyncSession

from app.db.session import get_db
from app.schemas import TerminologyEntryRead
from app.services.terminology_service import TerminologyService

router = APIRouter()
DbSession = Annotated[AsyncSession, Depends(get_db)]

@router.get("/search", response_model=list[TerminologyEntryRead])
async def search_terminology(
    db: DbSession,
    q: str = Query(min_length=1, max_length=200),
    limit: int = Query(default=10, ge=1, le=50),
) -> list[TerminologyEntryRead]:
    return await TerminologyService(db).search(q, limit=limit)

```

backend/app/core/__init__.py

```

"""Core configuration and shared utilities."""

```

backend/app/core/config.py

```

from functools import lru_cache

from pydantic import field_validator
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    app_name: str = "Railway QA Agent"
    app_env: str = "development"
    api_prefix: str = "/api"
    auto_create_tables: bool = True

```

```

database_url: str = "postgresql+asyncpg://deipss:change-me@***

qdrant_url: str = "***
qdrant_collection: str = "railway_knowledge"
qdrant_timeout_seconds: float = 8.0

llm_base_url: str | None = None
llm_api_key: str | None = None
llm_model: str | None = None
llm_temperature: float = 0.2
llm_timeout_seconds: float = 60.0

embedding_model: str = "BAAI/bge-m3"
embedding_base_url: str | None = None
embedding_api_key: str | None = None
embedding_timeout_seconds: float = 60.0
embedding_batch_size: int = 16
reranker_model: str | None = None
rag_vector_enabled: bool = True
retrieval_limit: int = 6

cors_origins: list[str] = [
    "***
    "***
    "***
    "***
]

model_config = SettingsConfigDict(
    env_file=".env", ".env",
    env_file_encoding="utf-8",
    extra="ignore",
)

@field_validator("cors_origins", mode="before")
@classmethod
def split_cors_origins(cls, value: str | list[str]) -> list[str] | str:
    if isinstance(value, str):
        return [origin.strip() for origin in value.split(",") if origin.strip()]
    return value

@lru_cache
def get_settings() -> Settings:
    return Settings()

settings = get_settings()

```

backend/app/db/__init__.py

```

"""Database setup."""

```

backend/app/db/base.py

```

from sqlalchemy import MetaData
from sqlalchemy.orm import DeclarativeBase

NAMING_CONVENTION = {
    "ix": "ix_(column_0_label)s",
    "uq": "uq_(table_name)s_(column_0_name)s",
    "ck": "ck_(table_name)s_(constraint_name)s",
    "fk": "fk_(table_name)s_(column_0_name)s_(referred_table_name)s",
    "pk": "pk_(table_name)s",
}

```

```
class Base(DeclarativeBase):
    metadata = MetaData(naming_convention=NAMING_CONVENTION)
```

backend/app/db/session.py

```
from collections.abc import AsyncIterator

from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker, create_async_engine

from app.core.config import settings
from app.db.base import Base

engine = create_async_engine(settings.database_url, pool_pre_ping=True)
AsyncSessionLocal = async_sessionmaker(engine, expire_on_commit=False)

async def get_db() -> AsyncIterator[AsyncSession]:
    async with AsyncSessionLocal() as session:
        yield session

async def init_db() -> None:
    from app import models # noqa: F401

    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
```

backend/app/main.py

```
from collections.abc import AsyncIterator
from contextlib import asynccontextmanager

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import ORJSONResponse

from app.api.router import api_router
from app.core.config import settings
from app.db.session import init_db

@asynccontextmanager
async def lifespan(_: FastAPI) -> AsyncIterator[None]:
    if settings.auto_create_tables:
        await init_db()
    yield

def create_app() -> FastAPI:
    app = FastAPI(
        title=settings.app_name,
        version="0.1.0",
        default_response_class=ORJSONResponse,
        lifespan=lifespan,
    )

    app.add_middleware(
        CORSMiddleware,
        allow_origins=settings.cors_origins,
        allow_credentials=True,
        allow_methods=["*"],
        allow_headers=["*"],
    )

    app.include_router(api_router, prefix=settings.api_prefix)
```

```

@app.get("/")
async def root() -> dict[str, str]:
    return {"name": settings.app_name, "status": "ok"}

return app

app = create_app()

```

backend/app/models/__init__.py

```

from app.models.entities import (
    ChatMessage,
    ChatSession,
    RetrievalTrace,
    TerminologyEntry,
    User,
)

__all__ = [
    "ChatMessage",
    "ChatSession",
    "RetrievalTrace",
    "TerminologyEntry",
    "User",
]

```

backend/app/models/entities.py

```

import uuid
from datetime import datetime

from sqlalchemy import DateTime, ForeignKey, Index, String, Text, func
from sqlalchemy.dialects.postgresql import JSONB, UUID
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.db.base import Base

class User(Base):
    __tablename__ = "users"

    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    username: Mapped[str] = mapped_column(String(80), unique=True, index=True)
    display_name: Mapped[str | None] = mapped_column(String(120))
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        server_default=func.now(),
    )

    sessions: Mapped[list["ChatSession"]] = relationship(back_populates="user")

class ChatSession(Base):
    __tablename__ = "chat_sessions"

    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    user_id: Mapped[uuid.UUID | None] = mapped_column(
        UUID(as_uuid=True),
        ForeignKey("users.id", ondelete="SET NULL"),
        index=True,
    )
    title: Mapped[str] = mapped_column(String(160), default="New conversation")
    language: Mapped[str] = mapped_column(String(16), default="auto")
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),

```

```

        server_default=func.now(),
    )
    updated_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        server_default=func.now(),
        onupdate=func.now(),
    )

    user: Mapped[User | None] = relationship(back_populates="sessions")
    messages: Mapped[list["ChatMessage"]] = relationship(
        back_populates="session",
        cascade="all, delete-orphan",
        order_by="ChatMessage.created_at",
    )

class ChatMessage(Base):
    __tablename__ = "chat_messages"

    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    session_id: Mapped[uuid.UUID] = mapped_column(
        UUID(as_uuid=True),
        ForeignKey("chat_sessions.id", ondelete="CASCADE"),
        index=True,
    )
    role: Mapped[str] = mapped_column(String(24), index=True)
    content: Mapped[str] = mapped_column(Text)
    language: Mapped[str] = mapped_column(String(16), default="auto")
    message_metadata: Mapped[dict | None] = mapped_column(JSONB)
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        server_default=func.now(),
        index=True,
    )

    session: Mapped[ChatSession] = relationship(back_populates="messages")
    retrieval_trace: Mapped["RetrievalTrace | None"] = relationship(
        back_populates="message",
        cascade="all, delete-orphan",
    )

class RetrievalTrace(Base):
    __tablename__ = "retrieval_traces"

    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    message_id: Mapped[uuid.UUID] = mapped_column(
        UUID(as_uuid=True),
        ForeignKey("chat_messages.id", ondelete="CASCADE"),
        unique=True,
        index=True,
    )
    query: Mapped[str] = mapped_column(Text)
    hits: Mapped[list[dict]] = mapped_column(JSONB, default=list)
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        server_default=func.now(),
    )

    message: Mapped[ChatMessage] = relationship(back_populates="retrieval_trace")

class TerminologyEntry(Base):
    __tablename__ = "terminology_entries"

    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    source_term: Mapped[str] = mapped_column(String(300), index=True)
    target_term: Mapped[str] = mapped_column(String(300), index=True)
    source_language: Mapped[str] = mapped_column(String(16), default="zh")
    target_language: Mapped[str] = mapped_column(String(16), default="en")
    category: Mapped[str | None] = mapped_column(String(120))

```

```

definition: Mapped[str | None] = mapped_column(Text)
aliases: Mapped[List[str] | None] = mapped_column(JSONB)
source_file: Mapped[str | None] = mapped_column(String(500))
created_at: Mapped[datetime] = mapped_column(
    DateTime(timezone=True),
    server_default=func.now(),
)

```

```

Index(
    "ix_terminology_entries_terms",
    TerminologyEntry.source_term,
    TerminologyEntry.target_term,
)

```

backend/app/rag/__init__.py

```

"""RAG orchestration components."""

```

backend/app/rag/docx_loader.py

```

from dataclasses import dataclass
from pathlib import Path

from docx import Document

@dataclass(frozen=True)
class DocumentBlock:
    text: str
    block_type: str
    heading: str | None
    index: int

@dataclass(frozen=True)
class DocumentChunk:
    text: str
    source_file: str
    heading: str | None
    chunk_index: int

def load_docx_blocks(path: Path) -> List[DocumentBlock]:
    document = Document(path)
    blocks: List[DocumentBlock] = []
    current_heading: str | None = None

    for paragraph in document.paragraphs:
        text = normalize_text(paragraph.text)
        if not text:
            continue

        style_name = (
            paragraph.style.name.lower()
            if paragraph.style and paragraph.style.name
            else ""
        )
        looks_like_heading = "heading" in style_name or text.startswith(("第", "Chapter"))
        block_type = "heading" if looks_like_heading else "paragraph"
        if block_type == "heading" and len(text) <= 120:
            current_heading = text

        blocks.append(
            DocumentBlock(
                text=text,

```

```

        block_type=block_type,
        heading=current_heading,
        index=len(blocks),
    )
)

for table in document.tables:
    for row in table.rows:
        cells = [normalize_text(cell.text) for cell in row.cells]
        cells = [cell for cell in cells if cell]
        if not cells:
            continue
        blocks.append(
            DocumentBlock(
                text=" | ".join(cells),
                block_type="table_row",
                heading=current_heading,
                index=len(blocks),
            )
        )

return blocks

def chunk_blocks(
    blocks: list[DocumentBlock],
    source_file: str,
    max_chars: int = 1200,
    overlap_chars: int = 160,
) -> list[DocumentChunk]:
    chunks: list[DocumentChunk] = []
    buffer: list[str] = []
    current_heading: str | None = None
    current_len = 0

    for block in blocks:
        next_len = len(block.text) + 1
        if buffer and current_len + next_len > max_chars:
            chunks.append(
                DocumentChunk(
                    text="\n".join(buffer),
                    source_file=source_file,
                    heading=current_heading,
                    chunk_index=len(chunks),
                )
            )
            overlap = "\n".join(buffer)[-overlap_chars:]
            buffer = [overlap] if overlap else []
            current_len = len(overlap)

        current_heading = block.heading or current_heading
        buffer.append(block.text)
        current_len += next_len

    if buffer:
        chunks.append(
            DocumentChunk(
                text="\n".join(buffer),
                source_file=source_file,
                heading=current_heading,
                chunk_index=len(chunks),
            )
        )

    return chunks

def normalize_text(text: str) -> str:
    return " ".join(text.replace("\u3000", " ").split())

```

backend/app/rag/embedding.py

```
from functools import lru_cache
from typing import Protocol

import httpx

from app.core.config import settings


class Embedder(Protocol):
    @property
    def dimension(self) -> int | None:
        ...

    def embed_texts(self, texts: list[str]) -> list[list[float]]:
        ...


class BgeM3Embedder:
    def __init__(self, model_name: str | None = None) -> None:
        try:
            from sentence_transformers import SentenceTransformer
        except ImportError as exc:
            raise RuntimeError(
                "Local BGE-M3 embedding requires sentence-transformers. "
                "For CPU-only installs, install a CPU PyTorch wheel first, then install "
                "sentence-transformers, or configure EMBEDDING_BASE_URL to use an "
                "OpenAI-compatible embedding service."
            ) from exc

        self.model_name = model_name or settings.embedding_model
        self.model = SentenceTransformer(self.model_name)
        self._dimension: int | None = None

    @property
    def dimension(self) -> int | None:
        return self._dimension

    def embed_texts(self, texts: list[str]) -> list[list[float]]:
        if not texts:
            return []
        embeddings = self.model.encode(
            texts,
            batch_size=settings.embedding_batch_size,
            normalize_embeddings=True,
            show_progress_bar=False,
        )
        vectors = [embedding.tolist() for embedding in embeddings]
        self._dimension = len(vectors[0]) if vectors else self._dimension
        return vectors


class OpenAICompatibleEmbedder:
    def __init__(self) -> None:
        if not settings.embedding_base_url:
            raise RuntimeError("EMBEDDING_BASE_URL is required for remote embeddings.")
        self._dimension: int | None = None

    @property
    def dimension(self) -> int | None:
        return self._dimension

    def embed_texts(self, texts: list[str]) -> list[list[float]]:
        if not texts:
            return []

        url = settings.embedding_base_url.rstrip("/") + "/embeddings"
        headers = {}
        if settings.embedding_api_key:
```

```

        headers["Authorization"] = f"Bearer {settings.embedding_api_key}"
        payload = {"model": settings.embedding_model, "input": texts}

        with httpx.Client(timeout=settings.embedding_timeout_seconds) as client:
            response = client.post(url, headers=headers, json=payload)
            response.raise_for_status()
            data = response.json()

        vectors = [
            item["embedding"]
            for item in sorted(data["data"], key=lambda item: item.get("index", 0))
        ]
        self._dimension = len(vectors[0]) if vectors else self._dimension
        return vectors

@lru_cache(maxsize=1)
def get_embedder() -> Embedder:
    if settings.embedding_base_url:
        return OpenAICompatibleEmbedder()
    return BgeM3Embedder(settings.embedding_model)

```

backend/app/rag/retriever.py

```

from sqlalchemy.ext.asyncio import AsyncSession

from app.core.config import settings
from app.rag.embedding import get_embedder
from app.rag.types import RetrievalHit
from app.rag.vector_store import QdrantVectorStore
from app.services.terminology_service import TerminologyService

class RailwayRetriever:
    def __init__(self, db: AsyncSession) -> None:
        self.db = db

    async def retrieve(self, query: str, limit: int | None = None) -> list[RetrievalHit]:
        max_hits = limit or settings.retrieval_limit
        terms = await TerminologyService(self.db).search(query, limit=min(max_hits, 5))
        hits: list[RetrievalHit] = []

        for term in terms:
            text = f"{term.source_term} = {term.target_term}"
            if term.definition:
                text += f"; {term.definition}"
            hits.append(
                RetrievalHit(
                    text=text,
                    source_file=term.source_file,
                    score=1.0,
                    kind="terminology",
                )
            )

        if settings.rag_vector_enabled:
            hits.extend(await self._retrieve_from_qdrant(query, limit=max_hits))

        return dedupe_hits(hits)[:max_hits]

    async def _retrieve_from_qdrant(self, query: str, limit: int) -> list[RetrievalHit]:
        try:
            query_vector = get_embedder().embed_texts([query])[0]
            store = QdrantVectorStore()
            try:
                return await store.search(query_vector=query_vector, limit=limit)
            finally:
                await store.close()
        except Exception:

```

```

        return []

def dedupe_hits(hits: List[RetrievalHit]) -> List[RetrievalHit]:
    deduped: List[RetrievalHit] = []
    seen: set[tuple[str | None, int | None, str]] = set()
    for hit in hits:
        key = (hit.source_file, hit.chunk_index, hit.text[:120])
        if key in seen:
            continue
        seen.add(key)
        deduped.append(hit)
    return deduped

```

backend/app/rag/terminology_extractor.py

```

import re
from collections.abc import Callable
from dataclasses import dataclass
from pathlib import Path

from docx import Document

from app.rag.docx_loader import normalize_text

CJK_PATTERN = re.compile(r"[\u4e00-\u9fff]")
PAIR_PATTERN = re.compile(
    r"(?P<zh>[\u4e00-\u9fff][\u4e00-\u9fffA-Za-z0-9 () \/\.\-\\s]{0,40}?)"
    r"\s+"
    r"(?P<en>[^\u4e00-\u9fff]+?)"
    r"(?=\s+[\u4e00-\u9fff]|\$)"
)

@dataclass(frozen=True)
class ExtractedTerm:
    source_term: str
    target_term: str
    source_language: str
    target_language: str
    category: str | None
    source_file: str

def extract_terms_from_docx(path: Path, limit: int | None = None) -> List[ExtractedTerm]:
    document = Document(path)
    category: str | None = None
    terms: List[ExtractedTerm] = []
    seen: set[tuple[str, str]] = set()

    def add_pairs(line: str) -> None:
        nonlocal terms
        for source_term, target_term in extract_pairs_from_line(line):
            key = (source_term.casefold(), target_term.casefold())
            if key in seen:
                continue
            seen.add(key)
            terms.append(
                ExtractedTerm(
                    source_term=source_term,
                    target_term=target_term,
                    source_language="zh",
                    target_language="en",
                    category=category,
                    source_file=path.name,
                )
            )

    if limit and len(terms) >= limit:
        return

```

```

for paragraph in document.paragraphs:
    line = normalize_text(paragraph.text)
    if not line:
        continue
    if is_category_line(line):
        category = line[:120]
        continue
    add_pairs(line)
    if limit and len(terms) >= limit:
        return terms

for table in document.tables:
    for row in table.rows:
        raw_cells = [cell.text for cell in row.cells]
        add_pairs_from_cells(raw_cells, add_pairs)
        if limit and len(terms) >= limit:
            return terms

return terms

def add_pairs_from_cells(raw_cells: list[str], add_pairs: Callable[[str], None]) -> None:
    cells = [cell for cell in raw_cells if normalize_text(cell)]
    if len(cells) >= 2:
        zh_lines = split_cell_lines(cells[0])
        en_lines = split_cell_lines(cells[1])
        if len(zh_lines) == len(en_lines) and len(zh_lines) > 1:
            for source_term, target_term in zip(zh_lines, en_lines, strict=True):
                if CJK_PATTERN.search(source_term) and not CJK_PATTERN.search(target_term):
                    add_pairs(f"{source_term} {target_term}")
            return

    add_pairs(normalize_text(" ".join(cells)))

def split_cell_lines(text: str) -> list[str]:
    return [normalize_text(line) for line in text.splitlines() if normalize_text(line)]

def extract_pairs_from_line(line: str) -> list[tuple[str, str]]:
    if should_skip_line(line):
        return []

    pairs: list[tuple[str, str]] = []
    for match in PAIR_PATTERN.finditer(line):
        source_term = normalize_source_term(match.group("zh"))
        target_term = normalize_target_term(match.group("en"))
        if is_valid_pair(source_term, target_term):
            pairs.append((source_term, target_term))
    return pairs

def normalize_source_term(term: str) -> str:
    term = normalize_text(term)
    term = re.sub(r"(?<=[\u4e00-\u9fff])\s+(?=[\u4e00-\u9fff])", "", term)
    return term.strip(" ,;,: ")

def normalize_target_term(term: str) -> str:
    term = normalize_text(term)
    return term.strip(" ,;,: ")

def is_category_line(line: str) -> bool:
    return (
        len(line) <= 80
        and CJK_PATTERN.search(line) is not None
        and any(token in line.lower() for token in ("vocabulary", "词汇"))
    )

```

```

def should_skip_line(line: str) -> bool:
    lowered = line.lower()
    skip_tokens = (
        "dictionary",
        "publishing",
        "出版社",
        "主编",
        "目录",
        "copyright",
    )
    return any(token in lowered for token in skip_tokens)

def is_valid_pair(source_term: str, target_term: str) -> bool:
    if not source_term or not target_term:
        return False
    if len(source_term) > 50 or len(target_term) > 120:
        return False
    if not CJK_PATTERN.search(source_term):
        return False
    if CJK_PATTERN.search(target_term):
        return False
    if not re.search(r"[A-Za-z]", target_term):
        return False
    return True

```

backend/app/rag/types.py

```

from dataclasses import dataclass

@dataclass(frozen=True)
class RetrievalHit:
    text: str
    source_file: str | None = None
    heading: str | None = None
    chunk_index: int | None = None
    score: float | None = None
    kind: str = "document"

    def as_dict(self) -> dict:
        return {
            "text": self.text,
            "source_file": self.source_file,
            "heading": self.heading,
            "chunk_index": self.chunk_index,
            "score": self.score,
            "kind": self.kind,
        }

```

backend/app/rag/vector_store.py

```

import uuid
from collections.abc import Iterable

from qdrant_client import AsyncQdrantClient
from qdrant_client.models import Distance, PointStruct, VectorParams

from app.core.config import settings
from app.rag.docx_loader import DocumentChunk
from app.rag.types import RetrievalHit

class QdrantVectorStore:
    def __init__(self) -> None:
        self.client = AsyncQdrantClient(
            url=settings.qdrant_url,

```

```

        timeout=settings.qdrant_timeout_seconds,
    )
    self.collection_name = settings.qdrant_collection

async def ensure_collection(self, vector_size: int, recreate: bool = False) -> None:
    exists = await self.client.collection_exists(self.collection_name)
    if exists and recreate:
        await self.client.delete_collection(self.collection_name)
        exists = False
    if not exists:
        await self.client.create_collection(
            collection_name=self.collection_name,
            vectors_config=VectorParams(size=vector_size, distance=Distance.COSINE),
        )

async def upsert_chunks(
    self,
    chunks: List[DocumentChunk],
    vectors: List[List[float]],
) -> None:
    points = [
        PointStruct(
            id=stable_point_id(chunk.source_file, chunk.chunk_index),
            vector=vector,
            payload={
                "text": chunk.text,
                "source_file": chunk.source_file,
                "heading": chunk.heading,
                "chunk_index": chunk.chunk_index,
                "doc_type": infer_doc_type(chunk.source_file),
            },
        )
        for chunk, vector in zip(chunks, vectors, strict=True)
    ]
    if points:
        await self.client.upsert(collection_name=self.collection_name, points=points)

async def search(self, query_vector: List[float], limit: int) -> List[RetrievalHit]:
    response = await self.client.query_points(
        collection_name=self.collection_name,
        query=query_vector,
        limit=limit,
        with_payload=True,
    )
    hits: List[RetrievalHit] = []
    for point in response.points:
        payload = point.payload or {}
        text = payload.get("text")
        if not isinstance(text, str) or not text:
            continue
        hits.append(
            RetrievalHit(
                text=text,
                source_file=as_optional_str(payload.get("source_file")),
                heading=as_optional_str(payload.get("heading")),
                chunk_index=as_optional_int(payload.get("chunk_index")),
                score=point.score,
                kind="document",
            )
        )
    return hits

async def close(self) -> None:
    await self.client.close()

def batched(items: List[DocumentChunk], size: int) -> Iterable[List[DocumentChunk]]:
    for index in range(0, len(items), size):
        yield items[index : index + size]

```

```

def stable_point_id(source_file: str, chunk_index: int) -> str:
    return str(uuid.uuid5(uuid.NAMESPACE_URL, f"{source_file}:{chunk_index}"))

def infer_doc_type(source_file: str) -> str:
    if "词汇" in source_file:
        return "terminology"
    if "规章" in source_file or "ECRL" in source_file:
        return "regulation"
    return "document"

def as_optional_str(value: object) -> str | None:
    return value if isinstance(value, str) else None

def as_optional_int(value: object) -> int | None:
    return value if isinstance(value, int) else None

```

backend/app/schemas/__init__.py

```

from app.schemas.chat import (
    ChatMessageRead,
    ChatRequest,
    ChatResponse,
    ChatSessionRead,
    Citation,
)
from app.schemas.health import HealthResponse
from app.schemas.terminology import TerminologyEntryRead

__all__ = [
    "ChatMessageRead",
    "ChatRequest",
    "ChatResponse",
    "ChatSessionRead",
    "Citation",
    "HealthResponse",
    "TerminologyEntryRead",
]

```

backend/app/schemas/chat.py

```

from datetime import datetime
from uuid import UUID

from pydantic import BaseModel, ConfigDict, Field

class Citation(BaseModel):
    source_file: str | None = None
    heading: str | None = None
    chunk_index: int | None = None
    text: str
    score: float | None = None
    kind: str = "document"

class ChatRequest(BaseModel):
    message: str = Field(min_length=1, max_length=8000)
    session_id: UUID | None = None
    user_id: UUID | None = None
    language: str | None = Field(default=None, pattern="^(auto|zh|en)$")

class ChatResponse(BaseModel):
    session_id: UUID

```

```

    message_id: UUID
    answer: str
    language: str
    citations: list[Citation] = []

class ChatMessageRead(BaseModel):
    id: UUID
    session_id: UUID
    role: str
    content: str
    language: str
    created_at: datetime

    model_config = ConfigDict(from_attributes=True)

class ChatSessionRead(BaseModel):
    id: UUID
    user_id: UUID | None
    title: str
    language: str
    created_at: datetime
    updated_at: datetime

    model_config = ConfigDict(from_attributes=True)

```

backend/app/schemas/health.py

```

from pydantic import BaseModel

class HealthResponse(BaseModel):
    status: str
    app: str
    environment: str

```

backend/app/schemas/terminology.py

```

from uuid import UUID

from pydantic import BaseModel, ConfigDict

class TerminologyEntryRead(BaseModel):
    id: UUID
    source_term: str
    target_term: str
    source_language: str
    target_language: str
    category: str | None = None
    definition: str | None = None
    aliases: list[str] | None = None
    source_file: str | None = None

    model_config = ConfigDict(from_attributes=True)

```

backend/app/services/__init__.py

```

"""Application services."""

```

backend/app/services/chat_service.py

```
from fastapi import HTTPException
from sqlalchemy.ext.asyncio import AsyncSession

from app.models import ChatMessage, ChatSession, RetrievalTrace
from app.rag.retriever import RailwayRetriever
from app.schemas import ChatRequest, ChatResponse, Citation
from app.services.language import detect_language
from app.services.llm_client import LLMClient

class ChatService:
    def __init__(self, db: AsyncSession) -> None:
        self.db = db
        self.llm = LLMClient()

    async def handle(self, request: ChatRequest) -> ChatResponse:
        language = (
            request.language
            if request.language and request.language != "auto"
            else detect_language(request.message)
        )
        session = await self._get_or_create_session(request, language)

        user_message = ChatMessage(
            session_id=session.id,
            role="user",
            content=request.message,
            language=language,
        )
        self.db.add(user_message)
        await self.db.flush()

        hits = await RailwayRetriever(self.db).retrieve(request.message)
        answer = await self._generate_answer(request.message, language, hits)

        assistant_message = ChatMessage(
            session_id=session.id,
            role="assistant",
            content=answer,
            language=language,
            message_metadata={"retrieval_hit_count": len(hits)},
        )
        self.db.add(assistant_message)
        await self.db.flush()

        self.db.add(
            RetrievalTrace(
                message_id=assistant_message.id,
                query=request.message,
                hits=[hit.as_dict() for hit in hits],
            )
        )

        await self.db.commit()
        await self.db.refresh(assistant_message)

        citations = [Citation(**hit.as_dict()) for hit in hits]
        return ChatResponse(
            session_id=session.id,
            message_id=assistant_message.id,
            answer=answer,
            language=language,
            citations=citations,
        )

    async def _get_or_create_session(self, request: ChatRequest, language: str) -> ChatSession:
        if request.session_id:
            session = await self.db.get(ChatSession, request.session_id)
```

```

        if session is None:
            raise HTTPException(status_code=404, detail="Chat session not found")
        return session

    title = request.message.strip().replace("\n", " ")[:80] or "New conversation"
    session = ChatSession(user_id=request.user_id, title=title, language=language)
    self.db.add(session)
    await self.db.flush()
    return session

async def _generate_answer(self, question: str, language: str, hits: list) -> str:
    context = "\n".join(f"- {hit.text}" for hit in hits)
    system_prompt = (
        "You are a bilingual railway support assistant. Answer only from the provided "
        "railway regulations and terminology context. If context is insufficient, say so."
    )
    user_prompt = f"Question:\n{question}\n\nContext:\n{context or '(no retrieved context)'}"

    try:
        llm_answer = await self.llm.complete(
            [
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": user_prompt},
            ]
        )
    except Exception:
        llm_answer = None

    if llm_answer:
        return llm_answer

    if language == "zh":
        if hits:
            joined = "\n".join(f"- {hit.text}" for hit in hits)
            return (
                f"已找到相关铁路术语或知识片段: \n{joined}\n\n"
                "当前未配置 LLM，暂不能生成完整规章解释。"
            )
        return (
            "问题已保存。当前后端对话与检索接口已可用，"
            "但尚未完成知识库向量入库或 LLM 配置，所以暂不能给出可靠的规章答案。"
        )

    if hits:
        joined = "\n".join(f"- {hit.text}" for hit in hits)
        return (
            f"Relevant railway terminology or knowledge snippets were found:\n{joined}\n\n"
            "LLM generation is not configured yet, "
            "so a full regulatory answer is not available."
        )
    return (
        "Your question has been saved. The chat and retrieval API is available, "
        "but vector ingestion or LLM configuration is not complete yet, "
        "so I cannot provide a reliable regulatory answer."
    )

```

backend/app/services/language.py

```

import re

ZH_PATTERN = re.compile(r"[\u4e00-\u9fff]")

def detect_language(text: str) -> str:
    return "zh" if ZH_PATTERN.search(text) else "en"

```

backend/app/services/llm_client.py

```
from __future__ import annotations

import httpx

from app.core.config import settings

class LLMClient:
    async def complete(self, messages: list[dict[str, str]]) -> str | None:
        if not settings.llm_base_url or not settings.llm_api_key or not settings.llm_model:
            return None

        url = settings.llm_base_url.rstrip("/") + "/chat/completions"
        headers = {"Authorization": f"Bearer {settings.llm_api_key}"}
        payload = {
            "model": settings.llm_model,
            "messages": messages,
            "temperature": settings.llm_temperature,
        }

        async with httpx.AsyncClient(timeout=settings.llm_timeout_seconds) as client:
            response = await client.post(url, headers=headers, json=payload)
            response.raise_for_status()
            data = response.json()

        return data["choices"][0]["message"]["content"]
```

backend/app/services/terminology_service.py

```
import re

from sqlalchemy import or_, select
from sqlalchemy.ext.asyncio import AsyncSession

from app.models import TerminologyEntry

CJK_RUN_PATTERN = re.compile(r"[\u4e00-\u9fff]{2,}")
EN_TOKEN_PATTERN = ***
ZH_QUESTION_WORDS = (
    "是什么意思",
    "是什么",
    "什么意思",
    "如何",
    "怎么",
    "怎样",
    "解释",
    "说明",
    "请问",
)
EN_STOPWORDS = {
    "what",
    "is",
    "are",
    "the",
    "a",
    "an",
    "of",
    "for",
    "to",
    "in",
    "and",
    "how",
    "should",
    "please",
    "explain",
}
```

```

class TerminologyService:
    def __init__(self, db: AsyncSession) -> None:
        self.db = db

    async def search(self, query: str, limit: int = 10) -> list[TerminologyEntry]:
        normalized = query.strip()
        if not normalized:
            return []

        search_terms = build_search_terms(normalized)
        if not search_terms:
            return []

        conditions = []
        for term in search_terms[:16]:
            pattern = f"%{term}%"
            conditions.extend(
                [
                    TerminologyEntry.source_term.ilike(pattern),
                    TerminologyEntry.target_term.ilike(pattern),
                ]
            )

        result = await self.db.execute(
            select(TerminologyEntry)
            .where(or_(*conditions))
            .limit(max(limit * 4, limit))
        )
        return rank_terms(normalized, list(result.scalars().all()))[:limit]

def build_search_terms(query: str) -> list[str]:
    terms: list[str] = [query]
    zh_query = query
    for word in ZH_QUESTION_WORDS:
        zh_query = zh_query.replace(word, " ")
    zh_query = re.sub(r"[.,!?:;()]", " ", zh_query)

    for run in CJK_RUN_PATTERN.findall(zh_query):
        terms.append(run)
        if len(run) > 4:
            terms.extend(run[index : index + 4] for index in range(0, len(run) - 3))
            terms.extend(run[index : index + 3] for index in range(0, len(run) - 2))

    english_tokens = [
        token.lower()
        for token in EN_TOKEN_PATTERN.findall(query)
        if token.lower() not in EN_STOPWORDS
    ]
    if english_tokens:
        terms.append(" ".join(english_tokens))
        terms.extend(english_tokens)
        for index in range(0, max(len(english_tokens) - 1, 0)):
            terms.append(" ".join(english_tokens[index : index + 2]))
        for index in range(0, max(len(english_tokens) - 2, 0)):
            terms.append(" ".join(english_tokens[index : index + 3]))

    return unique_terms(terms)

def unique_terms(terms: list[str]) -> list[str]:
    unique: list[str] = []
    seen: set[str] = set()
    for term in terms:
        normalized = term.strip().lower()
        if len(normalized) < 2 or normalized in seen:
            continue
        seen.add(normalized)
        unique.append(term.strip())
    return sorted(unique, key=len, reverse=True)

```

```
def rank_terms(query: str, entries: list[TerminologyEntry]) -> list[TerminologyEntry]:
    def score(entry: TerminologyEntry) -> tuple[int, int]:
        source = entry.source_term.lower()
        target = entry.target_term.lower()
        lowered_query = query.lower()
        exact_bonus = 10 if source in lowered_query or target in lowered_query else 0
        return exact_bonus, max(len(source), len(target))

    return sorted(entries, key=score, reverse=True)
```

backend/Dockerfile

```
FROM python:3.11-slim

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

WORKDIR /app

COPY pyproject.toml ./
RUN pip install --no-cache-dir .

COPY app ./app
COPY alembic ./alembic
COPY alembic.ini ./

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "****", "--port", "8000"]
```

backend/pyproject.toml

```
[project]
name = "railway-qa-agent-backend"
version = "0.1.0"
description = "FastAPI backend for a bilingual railway RAG assistant"
requires-python = ">=3.11"
dependencies = [
    "alembic>=1.16.0",
    "asynccpg>=0.30.0",
    "fastapi>=0.115.0",
    "httpx>=0.28.0",
    "orjson>=3.10.0",
    "pydantic>=2.10.0",
    "pydantic-settings>=2.8.0",
    "python-docx>=1.2.0",
    "python-dotenv>=1.1.0",
    "qdrant-client>=1.14.0",
    "sqlalchemy[asyncio]>=2.0.40",
    "uvicorn[standard]>=0.34.0",
]

[project.optional-dependencies]
dev = [
    "pytest>=8.3.0",
    "pytest-asyncio>=0.26.0",
    "ruff>=0.11.0",
]
rag = [
    "sentence-transformers>=3.3.0",
]

[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"
```

```
[tool.setuptools.packages.find]
include = ["app*"]

[tool.ruff]
line-length = 100
target-version = "py311"

[tool.ruff.lint]
select = ["E", "F", "I", "UP", "B"]

[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["."]
```

backend/railway_qa_agent_backend.egg-info/dependency_links.txt

backend/railway_qa_agent_backend.egg-info/requires.txt

```
alembic>=1.16.0
asyncpg>=0.30.0
fastapi>=0.115.0
httpx>=0.28.0
orjson>=3.10.0
pydantic>=2.10.0
pydantic-settings>=2.8.0
python-docx>=1.2.0
python-dotenv>=1.1.0
qdrant-client>=1.14.0
sqlalchemy[asyncio]>=2.0.40
uvicorn[standard]>=0.34.0

[dev]
pytest>=8.3.0
pytest-asyncio>=0.26.0
ruff>=0.11.0

[rag]
sentence-transformers>=3.3.0
```

backend/railway_qa_agent_backend.egg-info/SOURCES.txt

```
pyproject.toml
app/__init__.py
app/main.py
app/api/__init__.py
app/api/router.py
app/api/routes/__init__.py
app/api/routes/chat.py
app/api/routes/health.py
app/api/routes/terminology.py
app/core/__init__.py
app/core/config.py
app/db/__init__.py
app/db/base.py
app/db/session.py
app/models/__init__.py
app/models/entities.py
app/rag/__init__.py
app/rag/docx_loader.py
app/rag/embedding.py
app/rag/retriever.py
app/rag/terminology_extractor.py
app/rag/types.py
app/rag/vector_store.py
```

```
app/schemas/__init__.py
app/schemas/chat.py
app/schemas/health.py
app/schemas/terminology.py
app/services/__init__.py
app/services/chat_service.py
app/services/language.py
app/services/llm_client.py
app/services/terminology_service.py
railway_qa_agent_backend.egg-info/PKG-INFO
railway_qa_agent_backend.egg-info/SOURCES.txt
railway_qa_agent_backend.egg-info/dependency_links.txt
railway_qa_agent_backend.egg-info/requires.txt
railway_qa_agent_backend.egg-info/top_level.txt
tests/test_docx_loader.py
tests/test_language.py
tests/test_terminology_extractor.py
```

backend/railway_qa_agent_backend.egg-info/top_level.txt

```
app
```

backend/scripts/ingest_qdrant.py

```
import argparse
import asyncio
from pathlib import Path

from app.rag.docx_loader import DocumentChunk, chunk_blocks, load_docx_blocks
from app.rag.embedding import get_embedder
from app.rag.vector_store import QdrantVectorStore, batched

CORPUS_DIR = Path(__file__).resolve().parents[2] / "data" / "corpus" / "railway"

async def ingest(
    corpus_dir: Path,
    recreate: bool,
    limit: int | None,
    batch_size: int,
) -> int:
    chunks = collect_chunks(corpus_dir)
    if limit:
        chunks = chunks[:limit]
    if not chunks:
        return 0

    embedder = get_embedder()
    first_vector = embedder.embed_texts([chunks[0].text])[0]

    store = QdrantVectorStore()
    await store.ensure_collection(vector_size=len(first_vector), recreate=recreate)
    await store.upsert_chunks([chunks[0]], [first_vector])

    total = 1
    for batch in batched(chunks[1:], batch_size):
        vectors = embedder.embed_texts([chunk.text for chunk in batch])
        await store.upsert_chunks(batch, vectors)
        total += len(batch)
        print(f"upserted={total}/{len(chunks)}")

    await store.close()
    return total

def collect_chunks(corpus_dir: Path) -> List[DocumentChunk]:
    chunks: List[DocumentChunk] = []
    for path in sorted(corpus_dir.glob("*.docx")):
```

```

        blocks = load_docx_blocks(path)
        chunks.extend(chunk_blocks(blocks, source_file=path.name))
    return chunks

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Embed corpus chunks with BGE-M3 into Qdrant.")
    parser.add_argument("--corpus-dir", type=Path, default=CORPUS_DIR)
    parser.add_argument("--recreate", action="store_true")
    parser.add_argument("--limit", type=int, default=None)
    parser.add_argument("--batch-size", type=int, default=16)
    return parser.parse_args()

def main() -> None:
    args = parse_args()
    count = asyncio.run(
        ingest(
            corpus_dir=args.corpus_dir,
            recreate=args.recreate,
            limit=args.limit,
            batch_size=args.batch_size,
        )
    )
    print(f"ingested_chunks={count}")

if __name__ == "__main__":
    main()

```

backend/scripts/ingest_terminology.py

```

import argparse
import asyncio
from pathlib import Path

from sqlalchemy import delete

from app.db.session import AsyncSessionLocal
from app.models import TerminologyEntry
from app.rag.terminology_extractor import extract_terms_from_docx

DEFAULT_FILE = (
    Path(__file__).resolve().parents[2]
    / "data"
    / "corpus"
    / "railway"
    / "铁路中英文词汇（全）.docx"
)

async def ingest(path: Path, clear: bool, limit: int | None) -> int:
    terms = extract_terms_from_docx(path, limit=limit)
    async with AsyncSessionLocal() as session:
        if clear:
            await session.execute(
                delete(TerminologyEntry).where(TerminologyEntry.source_file == path.name)
            )
        session.add_all(
            [
                TerminologyEntry(
                    source_term=term.source_term,
                    target_term=term.target_term,
                    source_language=term.source_language,
                    target_language=term.target_language,
                    category=term.category,
                    source_file=term.source_file,
                )
                for term in terms
            ]
        )

```

```

        ]
    )
    await session.commit()
    return len(terms)

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Extract railway terminology into PostgreSQL.")
    parser.add_argument("--file", type=Path, default=DEFAULT_FILE)
    parser.add_argument("--clear", action="store_true")
    parser.add_argument("--limit", type=int, default=None)
    return parser.parse_args()

def main() -> None:
    args = parse_args()
    count = asyncio.run(ingest(args.file, clear=args.clear, limit=args.limit))
    print(f"ingested_terms={count}")

if __name__ == "__main__":
    main()

```

backend/scripts/preview_corpus.py

```

from pathlib import Path

from app.rag.docx_loader import chunk_blocks, load_docx_blocks

CORPUS_DIR = Path(__file__).resolve().parents[2] / "data" / "corpus" / "railway"

def main() -> None:
    for path in sorted(CORPUS_DIR.glob("*.docx")):
        blocks = load_docx_blocks(path)
        chunks = chunk_blocks(blocks, source_file=path.name)
        print(f"{path.name}: {len(blocks)} blocks, {len(chunks)} chunks")

if __name__ == "__main__":
    main()

```

backend/tests/test_docx_loader.py

```

from app.rag.docx_loader import DocumentBlock, chunk_blocks, normalize_text

def test_normalize_text() -> None:
    assert normalize_text(" 牵引\u3000供电   equipment ") == "牵引 供电 equipment"

def test_chunk_blocks_keeps_source_metadata() -> None:
    blocks = [
        DocumentBlock(text="Chapter 1", block_type="heading", heading="Chapter 1", index=0),
        DocumentBlock(text="A" * 50, block_type="paragraph", heading="Chapter 1", index=1),
        DocumentBlock(text="B" * 50, block_type="paragraph", heading="Chapter 1", index=2),
    ]

    chunks = chunk_blocks(blocks, source_file="rules.docx", max_chars=80, overlap_chars=5)

    assert len(chunks) == 2
    assert chunks[0].source_file == "rules.docx"
    assert chunks[0].heading == "Chapter 1"

```

backend/tests/test_language.py

```
from app.services.language import detect_language

def test_detect_language_zh() -> None:
    assert detect_language("牵引供电设备如何维护? ") == "zh"

def test_detect_language_en() -> None:
    assert detect_language("How should traction power equipment be maintained?") == "en"
```

backend/tests/test_terminology_extractor.py

```
from app.rag.terminology_extractor import extract_pairs_from_line

def test_extract_pairs_from_mixed_line() -> None:
    pairs = extract_pairs_from_line(
        "安全/安全性 safety 安全措施 safeguard 安全带 safety belt"
    )

    assert ("安全/安全性", "safety") in pairs
    assert ("安全措施", "safeguard") in pairs
    assert ("安全带", "safety belt") in pairs

def test_extract_pairs_normalizes_cjk_spaces() -> None:
    pairs = extract_pairs_from_line("标 引 indexing;designation 标 志 indicator;sign;marker")

    assert ("标引", "indexing;designation") in pairs
    assert ("标志", "indicator;sign;marker") in pairs
```

backend/tests/test_terminology_service.py

```
from app.services.terminology_service import build_search_terms

def test_build_search_terms_cleans_chinese_question() -> None:
    terms = build_search_terms("牵引供电是什么意思? ")

    assert "牵引供电" in terms

def test_build_search_terms_keeps_english_phrase() -> None:
    terms = build_search_terms("What is traction power supply equipment?")

    assert "traction power supply" in terms
    assert "traction" in terms
```

frontend/Dockerfile

```
FROM node:24-alpine

WORKDIR /app

COPY package.json package-lock.json* ./
RUN npm install

COPY . .

EXPOSE 5173
```

```
CMD ["npm", "run", "dev"]
```

frontend/index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <title>Railway QA Agent</title>
  </head>
  <body>
    <div id="app"></div>
    <script type="module" src="/src/main.ts"></script>
  </body>
</html>
```

frontend/package-lock.json

```
{
  "name": "railway-qa-agent-frontend",
  "version": "0.1.0",
  "lockfileVersion": 3,
  "requires": true,
  "packages": {
    "": {
      "name": "railway-qa-agent-frontend",
      "version": "0.1.0",
      "dependencies": {
        "@element-plus/icons-vue": "^2.3.1",
        "@lucide/vue": "^1.17.0",
        "element-plus": "^2.11.8",
        "pinia": "^2.3.1",
        "vue": "^3.5.13",
        "vue-router": "^4.5.0"
      },
      "devDependencies": {
        "@types/node": "^25.9.1",
        "@vitejs/plugin-vue": "^5.2.1",
        "typescript": "^5.7.2",
        "vite": "^6.0.5",
        "vue-tsc": "^2.2.0"
      }
    },
    "node_modules/@babel/helper-string-parser": {
      "version": "7.29.7",
      "resolved": "****",
      "integrity": "sha512-Pb5ijPrZ89GDH8223L4UP8i6QApWxs04RbPQJTewDV0/keR2E36MeKnyr6LYmUUvqRRI+Iv87SuF1W6ErINzYw==",
      "license": "MIT",
      "engines": {
        "node": ">=6.9.0"
      }
    },
    "node_modules/@babel/helper-validator-identifier": {
      "version": "7.29.7",
      "resolved": "****",
      "integrity": "sha512-qehxGkRj55h/ff8EMaJ+cYhyaKLHIXqYDn682wQD7Rnp9Uuj0QsHog2uS0r2vzr4pW+sXf90NeeayjcNaX3fFg==",
      "license": "MIT",
      "engines": {
        "node": ">=6.9.0"
      }
    },
    "node_modules/@babel/parser": {
      "version": "7.29.7",

```

```

    "arm64"
  ],
  "dev": true,
  "license": "MIT",
  "optional": true,
  "os": [
    "freebsd"
  ]
},
"node_modules/@rollup/rollup-freebsd-x64": {
  "version": "4.61.0",
  "resolved": "****",
  "integrity": "sha512-
VokPN3TSctKj65cyCNPuH4vMFA8awx0ot/0sp+4J7ZLNrkQEhXhawqPwajoi8H5ZFt61i0ugZJuTKXBJGJ17Q==",
  "cpu": [
    "x64"
  ],
  "dev": true,
  "license": "MIT",
  "optional": true,
  "os": [
    "freebsd"
  ]
},
"node_modules/@rollup/rollup-linux-arm-gnueabihf": {
  "version": "4.61.0",
  "resolved": "****",
  "integrity": "sha512-
DxH0P3wxm+Yzs/p3zrk9dw1rURu8p0Nv5+MRK/L70tnLNg5rLZraSBFZ8iUX0d9f2BlhJyEpIZUH/emjq4UJ4g==",
  "cpu": [
    "arm"
  ],
  "dev": true,
  "libc": [
    "glibc"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "linux"
  ]
},
"node_modules/@rollup/rollup-linux-arm-musleabihf": {
  "version": "4.61.0",
  "resolved": "****",
  "integrity": "sha512-
T6ZvMNe84kAz6TBWHC7hGAoEtzP1LWYw/AqayGWEF6uISt3Abk/st06LqRD9THd7Xz3NxzurUpzAuEAUbZf+nw==",
  "cpu": [
    "arm"
  ],
  "dev": true,
  "libc": [
    "musl"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "linux"
  ]
},
"node_modules/@rollup/rollup-linux-arm64-gnu": {
  "version": "4.61.0",
  "resolved": "****",
  "integrity": "sha512-
q/4hzvQkDs8b4jIBab1pnLiiM0ayTZsN2amBFPDzuyZxjEd4wDwx0UJFYM3c0ZzSf5Kw8fnWSprJzIBMkcR44Q==",
  "cpu": [
    "arm64"
  ],
  "dev": true,
  "libc": [
    "glibc"
  ]
}

```

```

    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ]
  },
  "node_modules/@rollup/rollup-linux-arm64-musl": {
    "version": "4.61.0",
    "resolved": "****",
    "integrity": "sha512-
vvYWX3akdEAY6km+9wAqFDnk6pQsbJKVnj7xawcvs/+fdLYBgP+U+Qq/LLfpIXYIZvZLHMAKD9HLdacSx/r3dw==",
    "cpu": [
      "arm64"
    ],
    "dev": true,
    "libc": [
      "musl"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ]
  },
  "node_modules/@rollup/rollup-linux-loong64-gnu": {
    "version": "4.61.0",
    "resolved": "****",
    "integrity": "sha512-
DePa5cq0xDp/Zp0V0XpeWaGew5iIv5DXp9NYbkX5PFQyWVX9184WCTh3hvr/7lhXo8ZVLbFLkz8+o/q1dU6gA==",
    "cpu": [
      "loong64"
    ],
    "dev": true,
    "libc": [
      "glibc"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ]
  },
  "node_modules/@rollup/rollup-linux-loong64-musl": {
    "version": "4.61.0",
    "resolved": "****",
    "integrity": "sha512-
LV8aWMB8UChgLMCEzs7RkN0GsH29RJalLQwm9fCIjLqwxQTiWAqNcc7wjBkH31hV0PU/yVxGYvrYsgfea2qw6g==",
    "cpu": [
      "loong64"
    ],
    "dev": true,
    "libc": [
      "musl"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ]
  },
  "node_modules/@rollup/rollup-linux-ppc64-gnu": {
    "version": "4.61.0",
    "resolved": "****",
    "integrity": "sha512-
QoNSnwQtatNu5grdBbsL0tt1uyL5EnS8DA8Mr3nLuMXbhdQNYhN+G4tBax7VCdxLKj8YJ0/4009Ho84jMnJtKA==",
    "cpu": [
      "ppc64"
    ],
    "dev": true,
    "libc": [

```

```
    "glibc"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "linux"
  ]
},
"node_modules/@rollup/rollup-linux-ppc64-musl": {
  "version": "4.61.0",
  "resolved": "****",
  "integrity": "sha512-/zZp5MKapIIApE8trN8qLGNSiRN9TUoaUZ1cmVu4XnVdd5LQLOXTtyi+vtfUbNnT3iyjzpPqYeKXmvJ+gJGYWw==",
  "cpu": [
    "ppc64"
  ],
  "dev": true,
  "libc": [
    "musl"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "linux"
  ]
},
"node_modules/@rollup/rollup-linux-riscv64-gnu": {
  "version": "4.61.0",
  "resolved": "****",
  "integrity": "sha512-RbrzcD3aJ1k3UbtMRRBNwojdVVyXjuVAFTfn/xPa6EEL6GE9Sm/akPgFTb9aAC9pMKGJ6CtWxaGrqWcabH+ySg==",
  "cpu": [
    "riscv64"
  ],
  "dev": true,
  "libc": [
    "glibc"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "linux"
  ]
},
"node_modules/@rollup/rollup-linux-riscv64-musl": {
  "version": "4.61.0",
  "resolved": "****",
  "integrity": "sha512-ZF+onDsBso8PJf1XaG9LB+09RnBpKGnY60rzC4CSHrtC1jb6jWLTkk4bRqdoCXHd22gyr2hiYmEAm8Wns/BOCw==",
  "cpu": [
    "riscv64"
  ],
  "dev": true,
  "libc": [
    "musl"
  ],
  "license": "MIT",
  "optional": true,
  "os": [
    "linux"
  ]
},
"node_modules/@rollup/rollup-linux-s390x-gnu": {
  "version": "4.61.0",
  "resolved": "****",
  "integrity": "sha512-Atk0aSIk5Zx2Wuh9dgRQgLP0Koc8h0eYpbWryMXyk8G8/HmPkwPPkMqIIDhrXHHYqfUzSJA/I7IWSBv8xSmRBA==",
  "cpu": [
    "s390x"
  ],
  "dev": true,
```

```
    "libc": [
      "glibc"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ]
  },
  "node_modules/@rollup/rollup-linux-x64-gnu": {
    "version": "4.61.0",
    "resolved": "****",
    "integrity": "sha512-0uM0cf3eZ5K+K4cYHkdxShFMPLPXC0dfDFEFn9dNYAEE2cVvm0fH7zFgRVoDgmtQ1m9k5q7qfrHzyMAubKYUA==",
    "cpu": [
      "x64"
    ],
    "dev": true,
    "libc": [
      "glibc"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ]
  },
  "node_modules/@rollup/rollup-linux-x64-musl": {
    "version": "4.61.0",
    "resolved": "****",
    "integrity": "sha512-mvFtE4A/t/7hRJ7X80zmu8FsIkAUat2nzl12pgU337BRmq87AQUJztwHz2Zv5/tjo9/C95E66CK03SI/ToEDJw==",
    "cpu": [
      "x64"
    ],
    "dev": true,
    "libc": [
      "musl"
    ],
    "license": "MIT",
    "optional": true,
    "os": [
      "linux"
    ]
  },
  "node_modules/@rollup/rollup-openbsd-x64": {
    "version": "4.61.0",
    "resolved": "****",
    "integrity": "sha512-z9b9+aTxvt8n2rNltMPvyaUfB8NJ+CVy0rGK/MdIKHx7B+LXmZpm/XbRsU7Rpf3fRqJ2uS6mBJiJveCtq8LHDg==",
    "cpu": [
      "x64"
    ],
    "dev": true,
    "license": "MIT",
    "optional": true,
    "os": [
      "openbsd"
    ]
  },
  "node_modules/@rollup/rollup-openharmony-arm64": {
    "version": "4.61.0",
    "resolved": "****",
    "integrity": "sha512-jXaXfQKMeHs0c+g8R6oo33RRC6w07G9jDBxAE5eAKX7m0cCbZLoYIPNhF69WL+P909IWHF040JgPi1ML2qkt7w==",
    "cpu": [
      "arm64"
    ],
    "dev": true,
    "license": "MIT",
    "optional": true,
```

```
    "os": [
      "openharmoney"
    ]
  },
  "node_modules/@rollup/rollup-win32-arm64-msvc": {
    "version": "4.61.0",
    "resolved": "****",
    "integrity": "sha512-
0XNwVFocS2IA4+QplhTZ2a+8hPZR7T8KuoZsNmJkK8y7cp83StHvGksfHzPG3wczWTczyWHVQuqe iTubjiyBg==",
    "cpu": [
      "arm64"
    ],
    "dev": true,
    "license": "MIT",
    "optional": true,
    "os": [
      "win32"
    ]
  },
  "node_modules/@rollup/rollup-win32-ia32-msvc": {
    "version": "4.61.0",
    "resolved": "****",
    "integrity": "sha512-
ALAbNtB0637LxSldqV43z0FfXoGfL2TW1DgAg/bs7aQswFbDewz2Sj3BUhiGfb0VtW571xbc9p+REdxhyN/Eg==",
    "cpu": [
      "ia32"
    ],
    "dev": true,
    "license": "MIT",
    "optional": true,
    "os": [
      "win32"
    ]
  },
  "node_modules/@rollup/rollup-win32-x64-gnu": {
    "version": "4.61.0",
    "resolved": "****",
    "integrity": "sha512-
QRSrQXyJ1M4tjNXdR0/G/IgV6LzfQQJYBjLWIEYkY2Xs86DRL/iEpQ4bLMDjJxSL7n19eDKKXmg0AmuBVYy8pQ==",
    "cpu": [
      "x64"
    ],
    "dev": true,
    "license": "MIT",
    "optional": true,
    "os": [
      "win32"
    ]
  },
  "node_modules/@rollup/rollup-win32-x64-msvc": {
    "version": "4.61.0",
    "resolved": "****",
    "integrity": "sha512-
tkuFxbvK0/HlGd0VsINF6vHSYH8AF8W0TcNxKDK6JZmrehngFj78pToc8iemtnvwiLDjs2G/qSzYFhe9U8q+fw==",
    "cpu": [
      "x64"
    ],
    "dev": true,
    "license": "MIT",
    "optional": true,
    "os": [
      "win32"
    ]
  },
  "node_modules/@types/estree": {
    "version": "1.0.9",
    "resolved": "****",
    "integrity": "sha512-
GhdPgy1eL4/ImP05X05Uw4cw2/M93BCUmnEvWZNstLCzEKME4Fkk+YpoA50iHNQmoS7Cafb8Xa3Pya8m1Qrzeg==",
    "dev": true,
    "license": "MIT"
  }
```

```

    },
    "node_modules/@types/lodash": {
      "version": "4.17.24",
      "resolved": "****",
      "integrity": "sha512-
gIW7lQLZbue7LRSWEFqL49QJJWThrTFFeIMJdp3eH4tKoxm10vEPg02rm4wCCSHS0cL3/Fizimb35b7k8atwsQ==",
      "license": "MIT"
    },
    "node_modules/@types/lodash-es": {
      "version": "4.17.12",
      "resolved": "****",
      "integrity": "sha512-
ONgftHUCv4v34VhXm8QBSftKVXtbkBG3ViCjs6+eJ5a6y6Mi/jiFGPc1sC7QK+9BFhWrURE3EOggmWaSxL90zQ==",
      "license": "MIT",
      "dependencies": {
        "@types/lodash": "*"
      }
    },
    "node_modules/@types/node": {
      "version": "25.9.1",
      "resolved": "****",
      "integrity": "sha512-
xfrLY7UD5rMJk3ZVJP8BNzS28J36YJg+xp+LPXV1TdWxr8uMH5A860QNxYDQe/yLDSgjxE52Q9Vn07p75tJxg==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "undici-types": ">=7.24.0 <7.24.7"
      }
    },
    "node_modules/@types/web-bluetooth": {
      "version": "0.0.21",
      "resolved": "****",
      "integrity": "sha512-
oIQLCGWtcFZy2JW77j9k8nHzA0pqMHLQeJDA48XXMWH6tjCQH55RCFz1bzsmR0yL6PUm+LLnUiI4BCn221inxA==",
      "license": "MIT"
    },
    "node_modules/@vitejs/plugin-vue": {
      "version": "5.2.4",
      "resolved": "****",
      "integrity": "sha512-
7Yx/SXS0cQq5Hiiv3orevHUFn+pmMB4cgbEkDYgnkUWb0WfeQ/wa2yFv6D5ICiCQOVpjA7vYDXrC7AG08yjDHA==",
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": "^18.0.0 || >=20.0.0"
      },
      "peerDependencies": {
        "vite": "^5.0.0 || ^6.0.0",
        "vue": "^3.2.25"
      }
    },
    "node_modules/@volar/language-core": {
      "version": "2.4.15",
      "resolved": "****",
      "integrity": "sha512-
3VHw+QZU0ZG9IuQmzT68IyN4hZN9GchGPhbD9+pa8CVv7rno0Zwo7T8weIbrRmihqy3ATpdfXFqRrFPVK6CA==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "@volar/source-map": "2.4.15"
      }
    },
    "node_modules/@volar/source-map": {
      "version": "2.4.15",
      "resolved": "****",
      "integrity": "sha512-
CPbMWLUN6hVZJYGcU/GSoHu4EnChiLaXI9n8c9la6RaI9W5JHX+NqG+GSQcB0JdC2FIBLdZJwGsKyBB71VLTg==",
      "dev": true,
      "license": "MIT"
    },
    "node_modules/@volar/typescript": {

```

```

    "version": "2.4.15",
    "resolved": "****",
    "integrity": "sha512-
2aZ8i0cqPGjXb4BhkMsPYDkkuc2ZQ6y0pqwAuNwUoncELqoy5fRg0QtLR9gB0g902iS0NAkvpIzs27geVyVdPg==",
    "dev": true,
    "license": "MIT",
    "dependencies": {
      "@volar/language-core": "2.4.15",
      "path-browserify": "^1.0.1",
      "vscode-uri": "^3.0.8"
    }
  },
  "node_modules/@vue/compiler-core": {
    "version": "3.5.35",
    "resolved": "****",
    "integrity": "sha512-
BUmHaR1J+0+CKZ9uJucdVTER1LHsdyvv7vG3eNRhK3CczEHeMd/LtsHAuD7PbrxvI2envCY2v7HI1vC1aBRzKw==",
    "license": "MIT",
    "dependencies": {
      "@babel/parser": "^7.29.3",
      "@vue/shared": "3.5.35",
      "entities": "^7.0.1",
      "estree-walker": "^2.0.2",
      "source-map-js": "^1.2.1"
    }
  },
  "node_modules/@vue/compiler-dom": {
    "version": "3.5.35",
    "resolved": "****",
    "integrity": "sha512-
k+bprkXxquhVajgTx5mUHuir7TwQzUK0WR40ng1ncAqQRPnrLNgGGqVEEHonTmLc8btHYVKmrP8s5Qyg0hvYA==",
    "license": "MIT",
    "dependencies": {
      "@vue/compiler-core": "3.5.35",
      "@vue/shared": "3.5.35"
    }
  },
  "node_modules/@vue/compiler-sfc": {
    "version": "3.5.35",
    "resolved": "****",
    "integrity": "sha512-
G5VPMcXTSywXBgtFOZ0nHKBxKSrwXUcvY1iaF5/hRcy7t0J6CH/d8ha9F4nzi00Fax1eLV0QHm7v4mQu68jydw==",
    "license": "MIT",
    "dependencies": {
      "@babel/parser": "^7.29.3",
      "@vue/compiler-core": "3.5.35",
      "@vue/compiler-dom": "3.5.35",
      "@vue/compiler-ssr": "3.5.35",
      "@vue/shared": "3.5.35",
      "estree-walker": "^2.0.2",
      "magic-string": "^0.30.21",
      "postcss": "^8.5.15",
      "source-map-js": "^1.2.1"
    }
  },
  "node_modules/@vue/compiler-ssr": {
    "version": "3.5.35",
    "resolved": "****",
    "integrity": "sha512-
rGhAeXgdM7/fftJGXT69rCCdTmjDewnFuUZfBQQHTdcEBewdT5HCGY60y2ytLjr9/Dsu7IntUi5z/w0h6Rjnzw==",
    "license": "MIT",
    "dependencies": {
      "@vue/compiler-dom": "3.5.35",
      "@vue/shared": "3.5.35"
    }
  },
  "node_modules/@vue/compiler-vue2": {
    "version": "2.7.16",
    "resolved": "****",
    "integrity": "sha512-
qYC3Psj9S/mfu9uVi5WvNZIzq+xnXMh0wbTFKKDD7b1lhpn71jXSFdTQ+wsIEk00NCd7VV2IMm70NL6tbQ86A==",

```

```

    "dev": true,
    "license": "MIT",
    "dependencies": {
      "de-indent": "^1.0.2",
      "he": "^1.2.0"
    }
  },
  "node_modules/@vue/devtools-api": {
    "version": "6.6.4",
    "resolved": "****",
    "integrity": "sha512-sGhTPMuXqZ1rV0k32RyLztWkFXTRhuS7vgAKv0zjqk8gbsHkJ7x7FF+jbySxt7tW0bEJwyKaHMiKv/WGDiQm8g==",
    "license": "MIT"
  },
  "node_modules/@vue/language-core": {
    "version": "2.2.12",
    "resolved": "****",
    "integrity": "sha512-IsGljWbKGU1mZpBPN+BvPAdr55YPkj2nB/TBNGNC32Vy2qLG25DYu/NBN2vNtZqdRbTRjaoYrahLrToim2NanA==",
    "dev": true,
    "license": "MIT",
    "dependencies": {
      "@volar/language-core": "2.4.15",
      "@vue/compiler-dom": "^3.5.0",
      "@vue/compiler-vue2": "^2.7.16",
      "@vue/shared": "^3.5.0",
      "alien-signals": "^1.0.3",
      "minimatch": "^9.0.3",
      "muggle-string": "^0.4.1",
      "path-browserify": "^1.0.1"
    },
    "peerDependencies": {
      "typescript": "*"
    },
    "peerDependenciesMeta": {
      "typescript": {
        "optional": true
      }
    }
  },
  "node_modules/@vue/reactivity": {
    "version": "3.5.35",
    "resolved": "****",
    "integrity": "sha512-tVc+SsHConvH/Lz64qq1pP3rYArBmK42xonovEcXy745QtvctZodG/zhq54P5dr38cVuw25d27cPNRdLmIdpGQ==",
    "license": "MIT",
    "dependencies": {
      "@vue/shared": "3.5.35"
    }
  },
  "node_modules/@vue/runtime-core": {
    "version": "3.5.35",
    "resolved": "****",
    "integrity": "sha512-A/xFNX9loIcWdygeQuNCfKuh0CoYBzxhqEMNah5TSFg9Z53DrFYEN2qi5CU9necjM10WYegYREUTHmXTmhfXtg==",
    "license": "MIT",
    "dependencies": {
      "@vue/reactivity": "3.5.35",
      "@vue/shared": "3.5.35"
    }
  },
  "node_modules/@vue/runtime-dom": {
    "version": "3.5.35",
    "resolved": "****",
    "integrity": "sha512-odrJ1C391dbGnyDRh8U+rnP7J2amIEzfMk5vXy7xi3aZhEXofTvpi0T4HJb6jLqQZTNPR5MPHSB3RHNkIORA==",
    "license": "MIT",
    "dependencies": {
      "@vue/reactivity": "3.5.35",
      "@vue/runtime-core": "3.5.35",
      "@vue/shared": "3.5.35",

```

```

    "csstype": "^3.2.3"
  },
  "node_modules/@vue/server-renderer": {
    "version": "3.5.35",
    "resolved": "****",
    "integrity": "sha512-NkebS0Yd8B97wi80Qc03HqzZSLymJi/aWsn/7h740SVhRTm6qGs3Jp3e0rCXymWwSLKeRrnlIug+ilYoHBmQDA==",
    "license": "MIT",
    "dependencies": {
      "@vue/compiler-ssr": "3.5.35",
      "@vue/shared": "3.5.35"
    },
    "peerDependencies": {
      "vue": "3.5.35"
    }
  },
  "node_modules/@vue/shared": {
    "version": "3.5.35",
    "resolved": "****",
    "integrity": "sha512-zSbjL7gRXwks2ZQLRGcAjBtBXE0XW9Ddhn/HvSdrGkE2dqGnumzW8XtusRrxrE9LvqtiqDXQ+A60Hp6mvdYxfA==",
    "license": "MIT"
  },
  "node_modules/@vueuse/core": {
    "version": "14.3.0",
    "resolved": "****",
    "integrity": "sha512-aHfz47g0ZhMtTVHmIzMPvJy8ePhh0y68GY5bv110+5DVtZ+W7Bs0x+m61UNQqfrWyPztIHianWa3E2tib3NFIw==",
    "license": "MIT",
    "dependencies": {
      "@types/web-bluetooth": "^0.0.21",
      "@vueuse/metadata": "14.3.0",
      "@vueuse/shared": "14.3.0"
    },
    "funding": {
      "url": "****"
    },
    "peerDependencies": {
      "vue": "^3.5.0"
    }
  },
  "node_modules/@vueuse/metadata": {
    "version": "14.3.0",
    "resolved": "****",
    "integrity": "sha512-BwxmbAzwAVF50+MW57GXOUEV61nFBGnlBvrTqj49PqWJu3uw7hdu72ztXeZ33RdZtDY6k0+bfCAE1PCn88Tktw==",
    "license": "MIT",
    "funding": {
      "url": "****"
    }
  },
  "node_modules/@vueuse/shared": {
    "version": "14.3.0",
    "resolved": "****",
    "integrity": "sha512-bZpge9eSXwa4ToSiqJ7j6KRwhAsneMFoSz3LMWKQDkqimm3D/tbFlrklrs/IOqC8tEcYmXQZJ6N0UrhjBirVCg==",
    "license": "MIT",
    "funding": {
      "url": "****"
    },
    "peerDependencies": {
      "vue": "^3.5.0"
    }
  },
  "node_modules/alien-signals": {
    "version": "1.0.13",
    "resolved": "****",
    "integrity": "sha512-0Gj9yyTnJEttvzhTUwusc0vtqxq5vrhF7vL9oS0xJ2mK0ItPYP1/y+vCFebfxoEyAz0++1AIwJ5CMr+Fk3nDmg==",
    "dev": true,

```

```

    "license": "MIT"
  },
  "node_modules/async-validator": {
    "version": "4.2.5",
    "resolved": "****",
    "integrity": "sha512-7HhHjtERjqlNbZtqNqy2rckN/Sp00LmDliet+LP7k+eKZEjPk3DgyeU9LIXLdeLz0uBbbVp+9Qdow9wJWgwwfg==",
    "license": "MIT"
  },
  "node_modules/balanced-match": {
    "version": "1.0.2",
    "resolved": "****",
    "integrity": "sha512-3oSeU00TMV67hN1AmbXsK4yaqU7tjiHlbrRDZ0pH0KW9+CeX4bRAaX0Anxt0tx2MrpRpWwQaPwI LISEJhYU5Pw==",
    "dev": true,
    "license": "MIT"
  },
  "node_modules/brace-expansion": {
    "version": "2.1.1",
    "resolved": "****",
    "integrity": "sha512-3oSeU00TMV67hN1AmbXsK4yaqU7tjiHlbrRDZ0pH0KW9+CeX4bRAaX0Anxt0tx2MrpRpWwQaPwI LISEJhYU5Pw==",
    "dev": true,
    "license": "MIT",
    "dependencies": {
      "balanced-match": "^1.0.0"
    }
  },
  "node_modules/csstype": {
    "version": "3.2.3",
    "resolved": "****",
    "integrity": "sha512-z1HGKcYy2xA8AGQfwrn0PAY+PB7X/GSj3UVJW9qKyn43xWa+gLnXmU4qqLMRzWVLF8KusUX8T/0kCi0YpAIQ==",
    "license": "MIT"
  },
  "node_modules/dayjs": {
    "version": "1.11.21",
    "resolved": "****",
    "integrity": "sha512-98IT+H0ahAisibz/yjKbzu0BwYcjJ7BCLPzARYHiyEBmRz4fatF+KPJszeEHXsGYjUG234aH/c0jW1wwTbKUZlA==",
    "license": "MIT"
  },
  "node_modules/de-indent": {
    "version": "1.0.2",
    "resolved": "****",
    "integrity": "sha512-e/1zu3xH5MQryN2zdVaF00rdNLUBvWxzMbi+iNA6Bky7L1RoP8a2fIbRocyHcLXt/arDrrR6LL3TqFD9pMQTsg==",
    "dev": true,
    "license": "MIT"
  },
  "node_modules/element-plus": {
    "version": "2.14.1",
    "resolved": "****",
    "integrity": "sha512-UFnm1+BckNi+azkKJ7L32q1uXs9ekr99Z9pwTQPeDR05jqEWUwQq51ro4kZMvRANbjknX3Z7ukCZWti2T6Tr9A==",
    "license": "MIT",
    "dependencies": {
      "@ctrl/tinycolor": "^4.2.0",
      "@element-plus/icons-vue": "^2.3.2",
      "@floating-ui/dom": "^1.7.6",
      "@popperjs/core": "npm:@sxzz/popperjs-es@^2.11.8",
      "@types/lodash": "^4.17.24",
      "@types/lodash-es": "^4.17.12",
      "@vueuse/core": "14.3.0",
      "async-validator": "^4.2.5",
      "dayjs": "^1.11.20",
      "lodash": "^4.18.1",
      "lodash-es": "^4.18.1",
      "lodash-unified": "^1.0.3",
      "memoize-one": "^6.0.0",
      "normalize-wheel-es": "^1.2.0",
    }
  }

```

```

    "vue-component-type-helpers": "^3.3.1"
  },
  "peerDependencies": {
    "vue": "^3.3.7"
  }
},
"node_modules/entities": {
  "version": "7.0.1",
  "resolved": "****",
  "integrity": "sha512-TWrgLOFUTQH94YUyL1yT4uyavY5nNB5muff+RtWaqNVCAK408b5ZnnbNAUEWLTcpum9w6arT70i1XdQ4UeOPA==",
  "license": "BSD-2-Clause",
  "engines": {
    "node": ">=0.12"
  },
  "funding": {
    "url": "****"
  }
},
"node_modules/esbuild": {
  "version": "0.25.12",
  "resolved": "****",
  "integrity": "sha512-bbPBYrtZbkt60s6FiTLCTFxxv4tt3JKall1vRwshA3fdVztsLAatFaZobhkBC8/BrPetoa0oksYoKXoG4ryJg==",
  "dev": true,
  "hasInstallScript": true,
  "license": "MIT",
  "bin": {
    "esbuild": "bin/esbuild"
  },
  "engines": {
    "node": ">=18"
  },
  "optionalDependencies": {
    "@esbuild/aix-ppc64": "0.25.12",
    "@esbuild/android-arm": "0.25.12",
    "@esbuild/android-arm64": "0.25.12",
    "@esbuild/android-x64": "0.25.12",
    "@esbuild/darwin-arm64": "0.25.12",
    "@esbuild/darwin-x64": "0.25.12",
    "@esbuild/freebsd-arm64": "0.25.12",
    "@esbuild/freebsd-x64": "0.25.12",
    "@esbuild/linux-arm": "0.25.12",
    "@esbuild/linux-arm64": "0.25.12",
    "@esbuild/linux-ia32": "0.25.12",
    "@esbuild/linux-loong64": "0.25.12",
    "@esbuild/linux-mips64el": "0.25.12",
    "@esbuild/linux-ppc64": "0.25.12",
    "@esbuild/linux-riscv64": "0.25.12",
    "@esbuild/linux-s390x": "0.25.12",
    "@esbuild/linux-x64": "0.25.12",
    "@esbuild/netbsd-arm64": "0.25.12",
    "@esbuild/netbsd-x64": "0.25.12",
    "@esbuild/openbsd-arm64": "0.25.12",
    "@esbuild/openbsd-x64": "0.25.12",
    "@esbuild/openharmony-arm64": "0.25.12",
    "@esbuild/sunos-x64": "0.25.12",
    "@esbuild/win32-arm64": "0.25.12",
    "@esbuild/win32-ia32": "0.25.12",
    "@esbuild/win32-x64": "0.25.12"
  }
},
"node_modules/estree-walker": {
  "version": "2.0.2",
  "resolved": "****",
  "integrity": "sha512-Rfkk/Mp/DL7JVje3u18FxFuJQLTNR2q6QfMSMB7AvCBx91NGj/ba3kCfza0f6dVDbw7YLRf/nDrn7pQrCCyQ/w==",
  "license": "MIT"
},
"node_modules/fdir": {
  "version": "6.5.0",

```

```

    "resolved": "****",
    "integrity": "sha512-
tIbYtZbuc0s0BRGqPJkshJUyDl+SDH7dVM8gJy+ERp3WAUjLEFJE+02kanyHtwjWOnwrKYBiwAmM0p4kLJAnXg==",
    "dev": true,
    "license": "MIT",
    "engines": {
      "node": ">=12.0.0"
    },
    "peerDependencies": {
      "picomatch": "^3 || ^4"
    },
    "peerDependenciesMeta": {
      "picomatch": {
        "optional": true
      }
    }
  },
  "node_modules/fsevents": {
    "version": "2.3.3",
    "resolved": "****",
    "integrity": "sha512-
5x0DfX+fl7faATnagmWPpbFtwh/R77WmMMqqHGS65C3vvB0YHrgF+B1YmZ3441tMj5n63k0212XNoJwzLhffQw==",
    "dev": true,
    "hasInstallScript": true,
    "license": "MIT",
    "optional": true,
    "os": [
      "darwin"
    ],
    "engines": {
      "node": "^8.16.0 || ^10.6.0 || >=11.0.0"
    }
  },
  "node_modules/he": {
    "version": "1.2.0",
    "resolved": "****",
    "integrity": "sha512-
F/1DnUGPop0RZi0ni+CvrCgHQ5FyEAHRLSApuYWMmrBswN2Mn/7k+Gl38gJnR7yyDZk6WLXwiGod1JOWNDKGw==",
    "dev": true,
    "license": "MIT",
    "bin": {
      "he": "bin/he"
    }
  },
  "node_modules/lodash": {
    "version": "4.18.1",
    "resolved": "****",
    "integrity": "sha512-
dMInicTPVE8d1e5otfwmmjLxZoUpiVLweTdUsi/Caj/gfzzbLBCe5sRHV/AsjuCmxWrt2TNGSYuCeCq+0Q==",
    "license": "MIT"
  },
  "node_modules/lodash-es": {
    "version": "4.18.1",
    "resolved": "****",
    "integrity": "sha512-
J8xewKD/Gk220Zbhp0VSwcs60zhd95ESDwez0FuA3/099925PdHJ70FHNTGtajL3ALZkykD32HykiMo+BIBI8A==",
    "license": "MIT"
  },
  "node_modules/lodash-unified": {
    "version": "1.0.3",
    "resolved": "****",
    "integrity": "sha512-
WK9qSozx0XD7ZJQlPqSQT+om2ZfcT4y0+03FuzAHD0wF6S0L0090LRPDx3vhTTLZ8cFKpBn+I0cVXK6q0cILfQ==",
    "license": "MIT",
    "peerDependencies": {
      "@types/lodash-es": "*",
      "lodash": "*",
      "lodash-es": "*"
    }
  },
  "node_modules/magic-string": {

```

```
    "version": "0.30.21",
    "resolved": "****",
    "integrity": "sha512-
vd2F4YUyEXKGcLHoq+TEyCjxueSeHnFxyyJNp80yg0XV4vUhnDer/LvvLqM/arB5bXQN5K2/3oinyCRyx8T2CQ==",
    "license": "MIT",
    "dependencies": {
      "@jridgewell/sourcemap-codec": "^1.5.5"
    }
  },
  "node_modules/memoize-one": {
    "version": "6.0.0",
    "resolved": "****",
    "integrity": "sha512-
rkpe71W0N0c0Xz6QD0eJETuWAJGnJ9afsl1srmwPrI+yBCkge5EycXXbYRyvL29zZVUWQCY7InPRCv3GDxuZNw==",
    "license": "MIT"
  },
  "node_modules/minimatch": {
    "version": "9.0.9",
    "resolved": "****",
    "integrity": "sha512-
OBwBN9AL4dqmETlpS2zasx+vTeWcLWzkbLfZk7KTA5j3je00Nz/tRCnZomUyvNg83wL5Zv9Ss6HmJXAgL8R2Yg==",
    "dev": true,
    "license": "ISC",
    "dependencies": {
      "brace-expansion": "^2.0.2"
    }
  },
  "engines": {
    "node": ">=16 || 14 >=14.17"
  },
  "funding": {
    "url": "****"
  }
},
"node_modules/muggle-string": {
  "version": "0.4.1",
  "resolved": "****",
  "integrity": "sha512-
VNTrAak/Kh02i8dqqnqnAH0a3cYBwEZe9h+D5h/1ZqFSTEFHdM65LR7RoIqq3tBBYavs0XV84NoHXZ0AkPyqQ==",
  "dev": true,
  "license": "MIT"
},
"node_modules/nanoid": {
  "version": "3.3.12",
  "resolved": "****",
  "integrity": "sha512-
ZB9RH/39qqq5Vu6Y+NmUaFhQR6pp+M2Xt76XBnEwDaGcVAqhlvxrL3B2bKS5D3NH3QR76v3aSrKaF/Kiy7LEtQ==",
  "funding": [
    {
      "type": "github",
      "url": "****"
    }
  ],
  "license": "MIT",
  "bin": {
    "nanoid": "bin/nanoid.cjs"
  },
  "engines": {
    "node": "^10 || ^12 || ^13.7 || ^14 || >=15.0.1"
  }
},
"node_modules/normalize-wheel-es": {
  "version": "1.2.0",
  "resolved": "****",
  "integrity": "sha512-
Wj7+EJQ8mSuXr2iWfnujrimU35R2W4FAErEyTmJoJ7ucwTn2h0USSsRehMb5RSYkxXGTM7Y9QpvPmp++w5ftoJw==",
  "license": "BSD-3-Clause"
},
"node_modules/path-browserify": {
  "version": "1.0.1",
  "resolved": "****",
  "integrity": "sha512-
```

```

b7uo2UCU0YZcnF/3ID0luL0Ji/bafxa1xPe7ZPsammBSpjSWQkjNxLt635YGS2MiR9GjvuXCtz2emr3jbsz98g==",
  "dev": true,
  "license": "MIT"
},
"node_modules/picocolors": {
  "version": "1.1.1",
  "resolved": "****",
  "integrity": "sha512-
xH2snhtb5M9liqDsmEw56le376mTZkEX/jEb/RxNFyegNuL7eNsLCXP9FDj/Lcu0X8KEyMceP2ntpaHrDEVA==",
  "license": "ISC"
},
"node_modules/picomatch": {
  "version": "4.0.4",
  "resolved": "****",
  "integrity": "sha512-
QP88BAKvMam/3NxB6vj2o21R6MjxZUAd6nLwAS/pnGvN9IVLocLHxGYIzFhg6fUQ+5th6P4dv4eW9jX3DSIj7A==",
  "dev": true,
  "license": "MIT",
  "engines": {
    "node": ">=12"
  },
  "funding": {
    "url": "****"
  }
},
"node_modules/pinia": {
  "version": "2.3.1",
  "resolved": "****",
  "integrity": "sha512-
khULZSwT9xXCAtbbxFYBKDc/bWAGWJj0gvxEtWkTN7KRm66EeT1ZdZj6i2ceh9sP2Pzqsbc704r2yngBrxBVug==",
  "license": "MIT",
  "dependencies": {

```

frontend/package.json

```

{
  "name": "railway-qa-agent-frontend",
  "version": "0.1.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite --host ***",
    "build": "npm run typecheck && vite build",
    "preview": "vite preview --host ***",
    "typecheck": "vue-tsc --noEmit -p tsconfig.json"
  },
  "dependencies": {
    "@element-plus/icons-vue": "^2.3.1",
    "@lucide/vue": "^1.17.0",
    "element-plus": "^2.11.8",
    "pinia": "^2.3.1",
    "vue": "^3.5.13",
    "vue-router": "^4.5.0"
  },
  "devDependencies": {
    "@types/node": "^25.9.1",
    "@vitejs/plugin-vue": "^5.2.1",
    "typescript": "^5.7.2",
    "vite": "^6.0.5",
    "vue-tsc": "^2.2.0"
  }
}

```

frontend/src/api/client.ts

```

import type {
  ChatMessage,
  ChatRequest,

```

```

    ChatResponse,
    ChatSession,
    HealthResponse,
  } from '@types/chat'

const API_BASE_URL = import.meta.env.VITE_API_BASE_URL ?? '/api'

async function request<T>(path: string, init?: RequestInit): Promise<T> {
  const response = await fetch(`${API_BASE_URL}${path}`, {
    headers: {
      'Content-Type': 'application/json',
      ...(init?.headers ?? {}),
    },
    ...init,
  })

  if (!response.ok) {
    const detail = await response.text()
    throw new Error(detail || `Request failed with ${response.status}`)
  }

  return (await response.json()) as T
}

export function getHealth(): Promise<HealthResponse> {
  return request<HealthResponse>('/health')
}

export function sendChat(payload: ChatRequest): Promise<ChatResponse> {
  return request<ChatResponse>('/chat', {
    method: 'POST',
    body: JSON.stringify(payload),
  })
}

export function listSessions(): Promise<ChatSession[]> {
  return request<ChatSession[]>('/chat/sessions')
}

export function getSessionMessages(sessionId: string): Promise<ChatMessage[]> {
  return request<ChatMessage[]>(`/chat/sessions/${sessionId}`)
}

```

frontend/src/App.vue

```

<template>
  <RouterView />
</template>

```

frontend/src/components/MessageBubble.vue

```

<script setup lang="ts">
import type { ChatMessage } from '@types/chat'

defineProps<{
  message: ChatMessage
}>()
</script>

<template>
  <article class="message" :class="`message--${message.role}`">
    <div class="message__meta">
      <span>{{ message.role === 'user' ? 'User' : 'Agent' }}</span>
      <span v-if="message.pending">Saving</span>
    </div>
    <div class="message__content">{{ message.content }}</div>
  </article>
</template>

```

```

<details v-if="message.citations?.length" class="message__citations">
  <summary>Sources</summary>
  <ul>
    <li v-for="(citation, index) in message.citations" :key="`${citation.text}-${index}`">
      <div class="citation__source">
        {{ citation.kind }}
        <span v-if="citation.source_file"> • {{ citation.source_file }}</span>
      </div>
      <p>{{ citation.text }}</p>
    </li>
  </ul>
</details>
</article>
</template>

```

frontend/src/env.d.ts

```

/// <reference types="vite/client" />

interface ImportMetaEnv {
  readonly VITE_API_BASE_URL?: string
}

interface ImportMeta {
  readonly env: ImportMetaEnv
}

```

frontend/src/main.ts

```

import 'element-plus/dist/index.css'
import './styles/main.css'

import ElementPlus from 'element-plus'
import { createPinia } from 'pinia'
import { createApp } from 'vue'

import App from './App.vue'
import router from './router'

const app = createApp(App)

app.use(createPinia())
app.use(router)
app.use(ElementPlus)
app.mount('#app')

```

frontend/src/router.ts

```

import { createRouter, createWebHistory } from 'vue-router'

import ChatView from './views/ChatView.vue'

const router = createRouter({
  history: createWebHistory(),
  routes: [
    {
      path: '/',
      name: 'chat',
      component: ChatView,
    },
  ],
})

```

```
export default router
```

frontend/src/stores/chat.ts

```
import { defineStore } from 'pinia'

import { getHealth, getSessionMessages, listSessions, sendChat } from '@api/client'
import type { ChatMessage, ChatSession, HealthResponse } from '@types/chat'

interface ChatState {
  activeSessionId: string | null
  sessions: ChatSession[]
  messages: ChatMessage[]
  health: HealthResponse | null
  loading: boolean
  sending: boolean
  error: string | null
}

export const useChatStore = defineStore('chat', {
  state: (): ChatState => ({
    activeSessionId: null,
    sessions: [],
    messages: [],
    health: null,
    loading: false,
    sending: false,
    error: null,
  }),
  actions: {
    async refreshHealth() {
      try {
        this.health = await getHealth()
      } catch (error) {
        this.health = null
        this.error = error instanceof Error ? error.message : 'Health check failed'
      }
    },
    async loadSessions() {
      this.loading = true
      try {
        this.sessions = await listSessions()
      } catch (error) {
        this.error = error instanceof Error ? error.message : 'Unable to load sessions'
      } finally {
        this.loading = false
      }
    },
    async selectSession(sessionId: string) {
      this.activeSessionId = sessionId
      this.loading = true
      try {
        this.messages = await getSessionMessages(sessionId)
      } catch (error) {
        this.error = error instanceof Error ? error.message : 'Unable to load messages'
      } finally {
        this.loading = false
      }
    },
    startNewSession() {
      this.activeSessionId = null
      this.messages = []
      this.error = null
    },
    async submitMessage(content: string) {
      const trimmed = content.trim()
      if (!trimmed || this.sending) return
    },
  },
})
```

```

    this.error = null
    this.sending = true
    const optimistic: ChatMessage = {
      id: crypto.randomUUID(),
      session_id: this.activeSessionId ?? 'pending',
      role: 'user',
      content: trimmed,
      language: 'auto',
      created_at: new Date().toISOString(),
      pending: true,
    }
    this.messages.push(optimistic)

    try {
      const response = await sendChat({
        message: trimmed,
        session_id: this.activeSessionId,
        language: 'auto',
      })

      this.activeSessionId = response.session_id
      optimistic.session_id = response.session_id
      optimistic.pending = false

      this.messages.push({
        id: response.message_id,
        session_id: response.session_id,
        role: 'assistant',
        content: response.answer,
        language: response.language,
        created_at: new Date().toISOString(),
        citations: response.citations,
      })
      await this.loadSessions()
    } catch (error) {
      optimistic.pending = false
      this.error = error instanceof Error ? error.message : 'Message failed'
    } finally {
      this.sending = false
    }
  },
},
})

```

frontend/src/styles/main.css

```

:root {
  color: #1f2933;
  background: #eef2f3;
  font-family:
    Inter, ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
  font-synthesis: none;
  text-rendering: optimizeLegibility;
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  min-width: 320px;
  min-height: 100vh;
}

button,
textarea {
  font: inherit;
}

```

```
}

button {
  cursor: pointer;
}

#app {
  min-height: 100vh;
}

.app-shell {
  display: grid;
  grid-template-columns: 300px minmax(0, 1fr);
  min-height: 100vh;
  background:
    linear-gradient(90deg, rgba(38, 102, 102, 0.08), transparent 36%),
    #eef2f3;
}

.sidebar {
  display: flex;
  flex-direction: column;
  gap: 18px;
  min-height: 100vh;
  padding: 22px;
  border-right: 1px solid #d8e0e3;
  background: #f8faf9;
}

.sidebar__brand,
.workspace__header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 16px;
}

.eyebrow {
  margin: 0 0 4px;
  color: #5c6972;
  font-size: 12px;
}

h1,
h2,
h3,
p {
  margin-top: 0;
}

h1 {
  margin-bottom: 0;
  color: #14213d;
  font-size: 24px;
  line-height: 1.2;
}

h2 {
  margin-bottom: 0;
  color: #14213d;
  font-size: 22px;
  line-height: 1.2;
}

.new-button,
.send-button,
.session-item {
  border: 1px solid transparent;
  border-radius: 8px;
}
```

```
.new-button,
.send-button {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 8px;
  min-height: 42px;
  padding: 0 16px;
  color: #ffffff;
  background: #1f7a75;
}

.new-button:hover,
.send-button:hover:not(:disabled) {
  background: #196a66;
}

.send-button:disabled {
  cursor: not-allowed;
  opacity: 0.48;
}

.session-list {
  display: flex;
  flex: 1;
  flex-direction: column;
  gap: 8px;
  min-height: 0;
  overflow-y: auto;
}

.session-item {
  display: grid;
  grid-template-columns: minmax(0, 1fr) auto;
  gap: 10px;
  width: 100%;
  padding: 12px;
  color: #24313a;
  text-align: left;
  background: #ffffff;
  border-color: #dde5e8;
}

.session-item:hover,
.session-item--active {
  border-color: #1f7a75;
  background: #eef8f6;
}

.session-item span {
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.session-item small {
  color: #8a5b17;
  font-size: 11px;
}

.workspace {
  display: grid;
  grid-template-rows: auto minmax(0, 1fr) auto auto;
  min-width: 0;
  min-height: 100vh;
}

.workspace__header {
  padding: 20px 28px;
  border-bottom: 1px solid #d8e0e3;
  background: rgba(255, 255, 255, 0.78);
}
```

```
}

.messages {
  display: flex;
  flex-direction: column;
  gap: 14px;
  min-height: 0;
  padding: 28px;
  overflow-y: auto;
}

.empty-state {
  align-self: center;
  width: min(520px, 100%);
  margin-top: 12vh;
  padding: 28px;
  border: 1px solid #d8e0e3;
  border-radius: 8px;
  background: #ffffff;
  text-align: center;
}

.empty-state h3 {
  margin-bottom: 8px;
  color: #14213d;
  font-size: 20px;
}

.empty-state p {
  margin-bottom: 0;
  color: #5c6972;
}

.message {
  width: min(820px, 100%);
  padding: 14px 16px;
  border: 1px solid #d8e0e3;
  border-radius: 8px;
  background: #ffffff;
}

.message--user {
  align-self: flex-end;
  border-color: #c7d7d5;
  background: #eef8f6;
}

.message--assistant {
  align-self: flex-start;
}

.message__meta {
  display: flex;
  justify-content: space-between;
  gap: 12px;
  margin-bottom: 8px;
  color: #5c6972;
  font-size: 12px;
}

.message__content {
  overflow-wrap: anywhere;
  white-space: pre-wrap;
  line-height: 1.65;
}

.message__citations {
  margin-top: 12px;
  color: #44515a;
}
```

```
.message__citations summary {
  color: #1f7a75;
  font-size: 13px;
  user-select: none;
}

.message__citations ul {
  display: grid;
  gap: 8px;
  padding-left: 18px;
}

.citation__source {
  color: #8a5b17;
  font-size: 12px;
}

.message__citations p {
  margin-bottom: 0;
  overflow-wrap: anywhere;
}

.composer {
  display: grid;
  grid-template-columns: minmax(0, 1fr) auto;
  gap: 12px;
  padding: 18px 28px 24px;
  border-top: 1px solid #d8e0e3;
  background: #f8faf9;
}

.composer__input .el-textarea__inner {
  border-radius: 8px;
}

.send-button {
  min-width: 112px;
  align-self: end;
}

.error-line {
  margin: 0 28px;
  padding: 10px 12px;
  color: #7f1d1d;
  border: 1px solid #f3b6b6;
  border-radius: 8px;
  background: #fff1f1;
}

@media (max-width: 860px) {
  .app-shell {
    grid-template-columns: 1fr;
  }

  .sidebar {
    min-height: auto;
    border-right: 0;
    border-bottom: 1px solid #d8e0e3;
  }

  .session-list {
    max-height: 190px;
  }

  .workspace {
    min-height: calc(100vh - 260px);
  }
}

@media (max-width: 640px) {
  .sidebar,
```

```
.workspace__header,  
.messages,  
.composer {  
  padding-right: 16px;  
  padding-left: 16px;  
}  
  
.composer {  
  grid-template-columns: 1fr;  
}  
  
.send-button {  
  width: 100%;  
}  
}
```

frontend/src/types/chat.ts

```
export interface Citation {  
  source_file?: string | null  
  heading?: string | null  
  chunk_index?: number | null  
  text: string  
  score?: number | null  
  kind: string  
}  
  
export interface ChatRequest {  
  message: string  
  session_id?: string | null  
  language?: 'auto' | 'zh' | 'en' | null  
}  
  
export interface ChatResponse {  
  session_id: string  
  message_id: string  
  answer: string  
  language: string  
  citations: Citation[]  
}  
  
export interface ChatSession {  
  id: string  
  user_id?: string | null  
  title: string  
  language: string  
  created_at: string  
  updated_at: string  
}  
  
export interface ChatMessage {  
  id: string  
  session_id: string  
  role: 'user' | 'assistant' | string  
  content: string  
  language: string  
  created_at: string  
  citations?: Citation[]  
  pending?: boolean  
}  
  
export interface HealthResponse {  
  status: string  
  app: string  
  environment: string  
}
```

frontend/src/views/ChatView.vue

```
<script setup lang="ts">
import { CirclePlus, SendHorizontal } from '@lucide/vue'
import { computed, nextTick, onMounted, ref, watch } from 'vue'

import MessageBubble from '@components/MessageBubble.vue'
import { useChatStore } from '@stores/chat'

const chat = useChatStore()
const draft = ref('')
const messagePane = ref<HTMLElement | null>(null)

const healthType = computed(() => (chat.health?.status === 'ok' ? 'success' : 'danger'))

async function submit() {
  const value = draft.value
  draft.value = ''
  await chat.submitMessage(value)
}

function scrollToBottom() {
  nextTick(() => {
    if (messagePane.value) {
      messagePane.value.scrollTop = messagePane.value.scrollHeight
    }
  })
}

watch(() => chat.messages.length, scrollToBottom)

onMounted(async () => {
  await Promise.all([chat.refreshHealth(), chat.loadSessions()])
})
</script>

<template>
  <div class="app-shell">
    <aside class="sidebar">
      <div class="sidebar__brand">
        <div>
          <p class="eyebrow">Railway QA</p>
          <h1>Bilingual Agent</h1>
        </div>
        <el-tag :type="healthType" size="small">API</el-tag>
      </div>

      <button class="new-button" type="button" @click="chat.startNewSession">
        <CirclePlus :size="18" />
        <span>New</span>
      </button>

      <nav class="session-list" aria-label="Chat sessions">
        <button
          v-for="session in chat.sessions"
          :key="session.id"
          class="session-item"
          :class="{ 'session-item--active': session.id === chat.activeSessionId }"
          type="button"
          @click="chat.selectSession(session.id)"
        >
          <span>{{ session.title }}</span>
          <small>{{ session.language.toUpperCase() }}</small>
        </button>
      </nav>
    </aside>

    <main class="workspace">
      <header class="workspace__header">
        <div>
```

```

    <p class="eyebrow">ECRL • Railway Terminology • Regulations</p>
    <h2>International Railway Support</h2>
  </div>
  <el-tag effect="plain">ZH / EN</el-tag>
</header>

<section ref="messagePane" class="messages" aria-live="polite">
  <div v-if="chat.messages.length === 0" class="empty-state">
    <h3>Railway regulatory QA</h3>
    <p>中文、English、术语、规章引用</p>
  </div>

  <MessageBubble v-for="message in chat.messages" :key="message.id" :message="message" />
</section>

<p v-if="chat.error" class="error-line">{{ chat.error }}</p>

<form class="composer" @submit.prevent="submit">
  <el-input
    v-model="draft"
    class="composer__input"
    :autosize="{ minRows: 2, maxRows: 6 }"
    resize="none"
    type="textarea"
    placeholder="输入铁路规章或术语问题"
    @keydown.meta.enter.prevent="submit"
    @keydown.ctrl.enter.prevent="submit"
  />
  <button class="send-button" type="submit" :disabled="chat.sending || !draft.trim()">
    <SendHorizontal :size="20" />
    <span>Send</span>
  </button>
</form>
</main>
</div>
</template>

```

frontend/tsconfig.json

```

{
  "compilerOptions": {
    "target": "ES2020",
    "useDefineForClassFields": true,
    "module": "ESNext",
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "skipLibCheck": true,
    "moduleResolution": "Bundler",
    "allowImportingTsExtensions": true,
    "resolveJsonModule": true,
    "isolatedModules": true,
    "noEmit": true,
    "jsx": "preserve",
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true,
    "baseUrl": ".",
    "paths": {
      "@/*": ["src/*"]
    }
  },
  "include": ["src/**/*.ts", "src/**/*.d.ts", "src/**/*.tsx", "src/**/*.vue"],
  "references": [{ "path": "./tsconfig.node.json" }]
}

```

frontend/tsconfig.node.json

```
{
  "compilerOptions": {
    "composite": true,
    "skipLibCheck": true,
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "allowSyntheticDefaultImports": true,
    "strict": true
  },
  "include": ["vite.config.ts"]
}
```

frontend/vite.config.ts

```
import vue from '@vitejs/plugin-vue'
import { fileURLToPath, URL } from 'node:url'
import { defineConfig } from 'vite'

export default defineConfig({
  plugins: [vue()],
  resolve: {
    alias: {
      '@': fileURLToPath(new URL('./src', import.meta.url)),
    },
  },
  server: {
    port: 5173,
    proxy: {
      '/api': {
        target: '***'
        changeOrigin: true,
      },
    },
  },
})
```

docker-compose.yml

```
services:
  postgres:
    image: postgres:16
    restart: unless-stopped
    environment:
      POSTGRES_USER: ${POSTGRES_USER:-deipss}
      POSTGRES_PASSWORD: ***
      POSTGRES_DB: ${POSTGRES_DB:-railway_qa_agent}
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-deipss} -d ${POSTGRES_DB:-railway_qa_agent}"]
      interval: 10s
      timeout: 5s
      retries: 5

  qdrant:
    image: qdrant/qdrant:v1.14.1
    restart: unless-stopped
    ports:
      - "6333:6333"
      - "6334:6334"
    volumes:
      - qdrant_data:/qdrant/storage
```

```
backend:
  build:
    context: ./backend
  restart: unless-stopped
  env_file:
    - .env
  environment:
    DATABASE_URL: ${DATABASE_URL}
    QDRANT_URL: ***
  depends_on:
    postgres:
      condition: service_healthy
    qdrant:
      condition: service_started
  ports:
    - "8000:8000"

frontend:
  build:
    context: ./frontend
  restart: unless-stopped
  environment:
    VITE_API_BASE_URL: ${VITE_API_BASE_URL:-/api}
  depends_on:
    - backend
  ports:
    - "5173:5173"

volumes:
  postgres_data:
  qdrant_data:
```

TODO.md

```
# TODO

更新时间：2026-06-03

## 当前已完成

- 已复制数据集到 `data/corpus/railway/`
  - `README.md`
  - `规章43：ECRL牵引供电设备运行维护管理办法（修订）_zh2en_transResult.docx`
  - `铁路中英文词汇（全）.docx`
- 已搭建前端：Vue 3 + TypeScript + Vite + Pinia + Element Plus
- 已搭建后端：FastAPI + SQLAlchemy async + PostgreSQL
- 已创建 PostgreSQL 数据库：`railway_qa_agent`
- 已抽取并入库铁路中英文术语：`31519` 条
- 已接通聊天接口的术语增强检索
- 已实现 DOCX 解析与切块模块
- 已实现 Qdrant 向量库接入代码和入库脚本
- 已实现 Embedding 抽象层
  - 支持 OpenAI-compatible embedding 服务
  - 支持本地 `sentence-transformers` 加载 `BAAI/bge-m3`
- 已实现 OpenAI-compatible LLM 调用配置
- 已补充 Docker Compose、Dockerfile、`.env.example` 和技术栈文档

## 已验证
```

cd backend

../.venv/bin/ruff check .

../.venv/bin/pytest

结果：通过，`8 passed`

```
cd frontend
npm run build
```

结果：通过

真实语料解析结果：

- 规章文档：`3052 blocks`，`436 chunks`
- 术语文档：`25233 blocks`，`977 chunks`
- 全部 DOCX 汇总切块：`1413 chunks`

术语入库验证：

- `terminology\_entries` 表：`31519` 条
- `牵引供电` 可命中：`traction power supply`

当前未完成

1. Qdrant 尚未实际启动

当前 `\*\*\*` 无服务监听。之前尝试 `docker compose up -d postgres qdrant` 失败，原因是当前用户没有访问 `/var/run/docker.sock` 的权限。

2. Qdrant 向量入库尚未实际执行

代码和脚本已完成，但需要先启动 Qdrant，再配置 embedding 服务或本地 BGE-M3 模型。

3. BGE-M3 embedding 尚未实际跑通

已尝试安装 `sentence-transformers`，但默认解析到 CUDA 版 PyTorch 依赖，体积很大，已中止。建议优先使用 OpenAI-compatible embedding 服务承载 `BAAI/bge-m3`。

4. LLM 尚未配置

当前聊天接口会保存消息、检索术语并返回降级答案。配置 `LLM\_BASE\_URL`、`LLM\_API\_KEY`、`LLM\_MODEL` 后即可调用真实模型。

恢复工作步骤

1. 准备环境变量

复制 `.env.example` 为 `.env`，注意密码中的 `@` 在 `DATABASE\_URL` 中要写成 `%40`。

```
cp .env.example .env
```

建议 `.env` 中至少包含：

```
POSTGRES_USER=deipss
POSTGRES_PASSWORD=***
POSTGRES_DB=railway_qa_agent
DATABASE_URL=postgresql+asyncpg://deipss:Java%40c1024@***
QDRANT_URL=***
QDRANT_COLLECTION=railway_knowledge
RAG_VECTOR_ENABLED=true
EMBEDDING_MODEL=BAAI/bge-m3
EMBEDDING_BASE_URL=
EMBEDDING_API_KEY=***
***
LLM_API_KEY=***
***
```

2. 启动基础服务

如果 Docker 权限已修复：

```
docker compose up -d postgres qdrant
```

如果只需要启动 Qdrant，也可以单独启动：

```
docker compose up -d qdrant
```

Qdrant 健康检查：

```
curl ***
```

3. 启动后端

```
cd backend
```

```
DATABASE_URL='postgresql+asyncpg://deipss:Java%40c1024@*** \
```

```
AUTO_CREATE_TABLES=true \
```

```
RAG_VECTOR_ENABLED=true \
```

```
../.venv/bin/uvicorn app.main:app --host *** --port 8000
```

健康检查：

```
curl ***
```

4. 启动前端

如果 `5173` 被占用，继续用 `5174`：

```
cd frontend
```

```
npm run dev -- --port 5174
```

访问：

5. 重新入库术语

术语已经入库过。需要重建时执行：

```
cd backend
```

```
DATABASE_URL='postgresql+asyncpg://deipss:Java%40c1024@*** \
```

```
../.venv/bin/python scripts/ingest_terminology.py --clear
```

6. 配置 BGE-M3 embedding

推荐方式：启动一个 OpenAI-compatible embedding 服务，模型为 `BAAI/bge-m3`，然后配置：

```
EMBEDDING_BASE_URL=***
```

```
EMBEDDING_MODEL=BAAI/bge-m3
```

再执行：

```
cd backend
```

```
DATABASE_URL='postgresql+asyncpg://deipss:Java%40c1024@*** \
```

```
EMBEDDING_BASE_URL='*** \
```

```
EMBEDDING_MODEL='BAAI/bge-m3' \  
../.venv/bin/python scripts/ingest_qdrant.py --recreate
```

备选方式：本地 CPU 模型。

```
cd backend  
../.venv/bin/pip install torch --index-url ***  
../.venv/bin/pip install sentence-transformers  
../.venv/bin/python scripts/ingest_qdrant.py --recreate
```

7. 配置 LLM

配置 OpenAI-compatible Chat Completions:

```
LLM_BASE_URL=***  
LLM_API_KEY=***  
LLM_MODEL=your-chat-model
```

配置后重启后端。

烟测命令

术语接口：

```
curl '***
```

聊天接口：

```
curl -s -X POST *** \  
-H 'Content-Type: application/json' \  
-d '{"message": "牵引供电是什么意思?", "language": "auto"}'
```

预期至少返回：

牵引供电 = traction power supply

重点代码位置

```
- `backend/scripts/ingest_terminology.py`  
- `backend/scripts/ingest_qdrant.py`  
- `backend/app/rag/terminology_extractor.py`  
- `backend/app/rag/embedding.py`  
- `backend/app/rag/vector_store.py`  
- `backend/app/rag/retriever.py`  
- `backend/app/services/chat_service.py`  
- `frontend/src/views/ChatView.vue`  
- `frontend/src/stores/chat.ts`
```